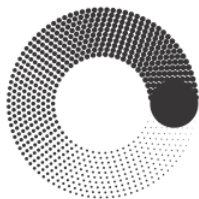


МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ



**Факультет Информационных технологий
кафедра Информатики и информационных технологий**

направление подготовки 09.002 «Информационные системы и технологии»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

БАКАЛАВРСКАЯ РАБОТА

Тема: Разработка сетевой составляющей и пошагового боевого режима в Rogue-like игре

Исполнитель Киселева Е. Р.

(Фамилия И.О.)

(Подпись)

Руководитель Арсентьев Д. А.

(Фамилия И.О., степень, звание)

(Подпись)

«ДОПУЩЕНО К ЗАЩИТЕ»

Зав. кафедрой ИиИТ:

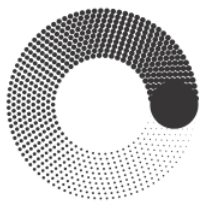
(Подпись)

к.т.н. Е.В. Булатников

Москва

2025

МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ



**Факультет Информационных технологий
кафедра Информатики и информационных технологий**

направление подготовки 09.002 «Информационные системы и технологии»

УТВЕРЖДАЮ

Зав. каф., к.т.н. Е.В. Булатников

« ____ » _____ 2025 г. _____

(Подпись)

ЗАДАНИЕ НА БАКАЛАВРСКУЮ РАБОТУ

Студент Киселева Екатерина Романовна группа 211-727

(Фамилия, Имя, Отчество)

Тема Разработка сетевой составляющей и пошагового боевого режима в Rogue-like игре

Утверждена приказом по Университету № 232-УД от «22» января 2025 г.

1. Срок представления работы к защите « 12 » июня 2025 г.

2. Исходные данные для выполнения работы: Техническое задание, техническая литература по теме ВКР

3. Содержание бакалаврской работы: Введение, аналитическая часть, конструкторская часть, практическая часть, заключение

4. Перечень графического материала Презентация

5. Дата выдачи задания: «22» января 2025 г.

6. Руководитель: _____ /Д.А. Арсентьев/
(Подпись) (И.О. Фамилия)

7. Задание к исполнению принял: _____ /Е. Р. Киселева/
(Подпись) (И.О. Фамилия)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ



Факультет Информационных технологий
кафедра Информатики и информационных технологий

направление подготовки 09.002 «Информационные системы и технологии»

КАЛЕНДАРНЫЙ ПЛАН

Студент Киселева Екатерина Романовна **группа** 211-727
(Фамилия, Имя, Отчество)

Тема ВКР: Разработка сетевой составляющей и пошагового боевого режима в Rogue-like игре

№ п/п	Наименование этапа ВКР	Срок выполнения этапа	Отметка о выполнении
	Исследование предметной области	26.07.2024-29.08.2024	Выполнено
	Разработка технического задания	30.07.2024-06.08.2024	Выполнено
	Эскизное проектирование	07.08.2024-13.09.2024	Выполнено
	Техническое проектирование	14.08.2024-27.10.2024	Выполнено
	Рабочее проектирование	28.09.2024-24.11.2024	Выполнено
	Разработка	25.09.2024-06.05.2025	Выполнено
	Тестирование	07.02.2025-10.05.2025	Выполнено
	Приемосдаточные испытания	21.04.2025-31.05.2025	Выполнено

Руководитель Арсентьев Д.А.
(Фамилия И.О., степень, звание) (Подпись)

Зав. каф. ИиИТ _____ /Е.В. Булатников/
(Подпись)

« ____ » _____ 2025 г.

Аннотация

В данной выпускной квалификационной работе рассматривается процесс создания игрового прототипа. Проект представляет собой модульную систему кооперативного Rogue-like с пошаговым боем, реализованную в среде Unity Engine для 2D игр.

Структура работы представлена введением, тремя главами, заключением, списком литературы и приложением.

Во введении обоснована актуальность темы проекта.

Первая глава посвящена изучению предметной области, системному анализу, поставленным цели и задачам.

Во второй главе описывается проектирование игрового проекта.

Третья глава посвящена процессу разработки и программной реализации.

Заключение содержит итогов проделанной работы и возможности дальнейшего развития проекта.

Abstract

This final qualification work considers the process of creating a game prototype. The project is a modular system of cooperative Rogue-like with turn-based combat, implemented in the Unity Engine environment for 2D games.

The structure of the work consists of an introduction, three chapters, a conclusion, bibliography, and appendix.

The introduction substantiates the relevance of the topic of the project.

The first chapter is dedicated to the study of subject area, systems analysis, goals, and objectives.

The second chapter describes the design of the game project.

The third chapter is dedicated to the process of development and software implementation.

The conclusion contains the results of the work done and the possibilities for further development of the project.

Реферат

Название: Разработка сетевой составляющей и пошагового боевого режима в Rogue-like игре.

Объем пояснительной записки составляет 62 страницы, включает 8 рисунков, 1 таблицу, 5 листингов, 4 приложения. При написании дипломной работы использовалось 14 источников.

Ключевые слова: Unity, C#, игровой проект, программный продукт, 2D игра, кооперативный режим, пошаговый бой, сетевая синхронизация.

Оглавление

Введение	8
Глава 1. Аналитическая часть.....	10
1.1 Цели и задачи	10
1.2 Анализ схожих проектов.....	11
1.3 Обзор выбранных технологий для разработки	13
1.1. Unity Engine	13
1.2 Visual Studio.....	14
1.3 Fork	15
Глава 2. Конструкторская часть	18
2.1 Разработка дизайн-документа	18
2.1.1. Общая информация о проекте	18
2.1.2. Определение основных особенности игры	19
2.1.3. Технические требования	20
2.2 Описание механик	21
2.2.1. Общая архитектура системы	21
2.2.2. Проектирование игровых механик	22
2.2.3. Алгоритмы и технические решения	24
Глава 3. Практическая часть	25
3.1. Сетевое взаимодействие.....	25
3.1.1. Управление игровыми сессиями	25
3.1.2. Синхронизация данных игроков	27
3.1.3. Сетевая синхронизация игровых состояний	28
3.1.4. Интерфейс сетевого взаимодействия	30
3.2 Пошаговая система боя	31
3.2.1. Архитектура боевой системы	32
3.2.2. Стадия предсказания	33
3.2.3. Стадия выбора действий игроков	34
3.2.4. Стадия выполнения действий игроков	35
3.2.5. Стадия действий босса	37
3.2.6. Управление очередностью ходов.....	38
3.3 Перемещение и навыки персонажей.....	39
3.3.1 Система перемещения персонажей.....	40
3.3.2 Управление игровым полем	41
3.3.3. Система навыков персонажей	43
3.3.4 Визуализация игрового поля и действий	44
3.3.5. Архитектура боевой системы	46

3.3 Перемещение и навыки персонажей.....	47
3.4.1 Базовая структура навыков игроков	48
3.4.2 Примеры реализации навыков игроков.....	49
3.4.3 Базовая структура действий босса	50
3.4.4 Примеры реализации действий босса.....	51
3.4.5 Управление событиями анимаций	53
3.4.6 Сетевая синхронизация и оптимизация.....	54
Заключение	56
Список литературы	57
Приложение А. Техническое задание	58
Приложение Б. Презентация.....	59
Приложение В. Листинг головоломки «Трубопровод»	60
Приложение Г. Листинг головоломки «Сбалансированные соединения»	61

Введение

Актуальность разрабатываемого проекта обусловлена устойчивым ростом популярности кооперативных видеоигр, в которых несколько игроков совместно взаимодействуют для достижения общих целей. В последние годы особую востребованность приобрел жанр Rogue-like, отличающийся процедурной генерацией уровней, высокой реиграбельностью и акцентом на тактический подход. Несмотря на разнообразие подобных проектов, ниша пошаговых кооперативных игр с асинхронными ролями участников остается слабо освоенной. По данным аналитической платформы SteamSpy за 2025 год, кооперативные Rogue-like проекты составляют менее 3 % от общего количества игр с кооперативной составляющей, в то время как средний пользовательский рейтинг таких игр превышает 78 %, а вовлеченность сохраняется на высоком уровне даже при ограниченной выборке.

Целью данной выпускной квалификационной работы является разработка кооперативной Rogue-like игры с пошаговым боем, в которой каждый участник команды управляет уникальным персонажем с определенной ролью (атака, защита, лечение, стратегия). Главный акцент проекта сделан на необходимость взаимодействия игроков, принятии тактических решений и координации действий для успешного прохождения уровней и победы над боссами.

Проект реализуется на платформе Unity 6 с использованием технологий Netcode for GameObjects и Unity Transport для создания и отладки сетевого взаимодействия. Это позволяет запускать игру с несколькими клиентами на одном устройстве, обеспечивая тестирование многопользовательского режима без необходимости подключения к внешнему серверу.

Игровая концепция включает в себя следующие и наличие командной синергии. Отдельное внимание уделяется балансу персонажей, чьи способности напрямую влияют на ход сражений и успех всей команды.

В рамках проекта реализуются такие функциональные модули, как система сетевого взаимодействия, менеджер игровых состояний, система способностей игроков и противников, модуль боя и внутриигровое взаимодействие между

клиентами. Проект проходит этапы жизненного цикла разработки — от анализа и планирования до тестирования.

Использование Unity Engine и языка программирования C# обеспечивает гибкость и масштабируемость разработки, что позволяет в дальнейшем расширить функциональность, интегрировать дополнительные режимы игры и переработать механику на основе пользовательского фидбека. Разработка направлена на создание высококачественного игрового продукта с устойчивым игровым циклом, ориентированного на командное взаимодействие и стратегическое мышление.

Глава 1. Аналитическая часть

1.1 Цели и задачи

Целью данного проекта является создание увлекательной кооперативной игры, требующей стратегического мышления, командной работы и тактического взаимодействия. Игра ориентирована на жанр Rogue-like с пошаговым боем, где каждый игрок выполняет определённую роль в команде, что позволяет формировать уникальные игровые сценарии и стимулирует взаимодействие между участниками. Для достижения этой цели необходимо решить следующие задачи:

1. Провести исследование предметной области тактических кооперативных игр, изучив ключевые механики и подходы к реализации.
2. Выполнить анализ аналогичных игровых продуктов для выявления их сильных и слабых сторон.
3. Спроектировать архитектуру программной системы, включающую модули управления игровым полем, системой навыков, действиями босса и визуализацией.
4. Разработать кооперативную 2D-игру, реализующую:
 - a. Систему перемещения персонажей по клеточному полю с учетом энергии и сетевой синхронизации.
 - b. Гибкую систему навыков игроков на основе ScriptableObject с визуальными и звуковыми эффектами.
 - c. Механику действий босса с динамическим выбором целей и анимацией через прозрачный спрайт.
 - d. Визуализацию игрового поля и действий с использованием пулов объектов для оптимизации.
 - e. Сетевую синхронизацию через Netcode for GameObjects для согласованности в многопользовательской игре.
 - f. Провести тестирование программного продукта для обеспечения стабильности и соответствия требованиям.

1.2 Анализ схожих проектов

В данном разделе рассматриваются успешные проекты, реализующие ключевые элементы, схожие с концепцией, разрабатываемой кооперативной Rogue-like игры с пошаговым боем и асинхронными ролями. Анализ позволяет выделить успешные игровые практики, влияющие на проектирование механик и интерфейса.

Тактический Roguelike «For The King» от студии IronOak Games, вышедший в 2018 году, совмещает элементы пошаговой стратегии, кооперативного геймплея и процедурной генерации уровней. Игроки управляют уникальными персонажами с индивидуальными наборами способностей и совместно принимают тактические решения. В игре реализована система ролей (танк, урон, поддержка), а взаимодействие между участниками — ключевой фактор выживания. При этом каждая боевая сцена требует обдуманных решений за ограниченное время, что повышает динамику пошагового боя.

Игра «Into the Breach» от Subset Games, выпущенная в 2018 году, демонстрирует глубокую пошаговую боевую систему, построенную на информации о намерениях противников. Игроку предоставляется ограниченное количество времени на обдумывание действий, что усиливает напряжение при принятии решений. Несмотря на одиночный режим, структура пошагового боя, необходимость учитывать синергию между бойцами и тактическое планирование вдохновляют реализацию аналогичных элементов в кооперативной среде.

Тактическая ролевая игра «Divinity: Original Sin 2» от Larian Studios отличается возможностью прохождения кампании в кооперативном режиме, где каждый игрок управляет собственным персонажем с уникальной ролью и влиянием на сюжет. Игра реализует систему ограниченного времени на ход в мультиплеере, стимулируя быструю и согласованную командную игру. Также интерес представляет гибкая система взаимодействия с окружением и

эффектами способностей, что может быть адаптировано в механике взаимодействия между ролями.

Игра «Darkest Dungeon» от Red Hook Studios вносит вклад в визуальную и тематическую составляющую разрабатываемого проекта. Пошаговая боевая система строится на взаимодействии персонажей с различными психологическими состояниями, а стилистика игры формирует атмосферу напряженности. Несмотря на одиночный режим, идеи по управлению стрессом и роли поддержки находят отражение в командной динамике.

Проведённый анализ позволил выделить следующие ориентиры для проекта:

- необходимость четкого распределения ролей с уникальными механиками;
- включение системы принятия решений в ограниченное время для поддержания темпа;
- акцент на тактическое взаимодействие между игроками;
- модульность архитектуры для масштабируемости и тестирования.

1.3 Обзор выбранных технологий для разработки

В данном разделе описываются особенности технологий, которые были выбраны для реализации игрового прототипа, их преимущества и применение в контексте разработки.

1.1. Unity Engine

В качестве основной среды разработки онлайн кооперативной игры был выбран игровой движок Unity Engine 6. Рабочая область данного инструмента продемонстрирована на рисунке 1.1.

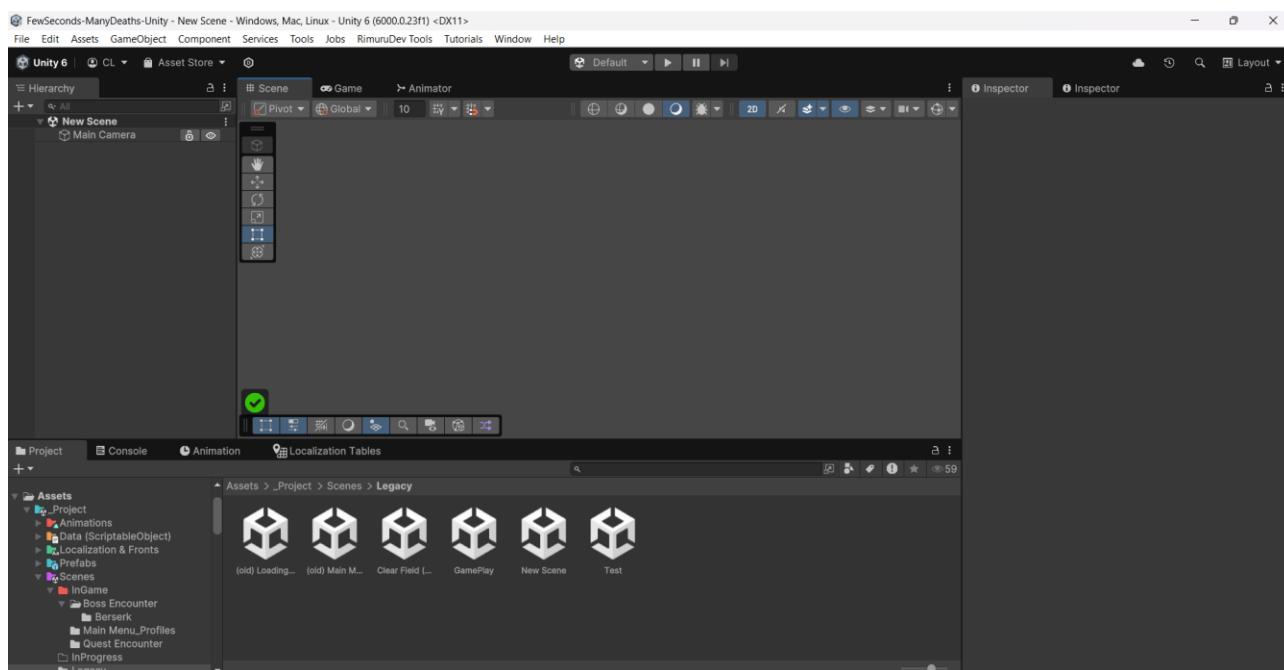


Рисунок 1.1 — Инструмент разработки Unity Engine 6

Данный игровой движок предоставляет широкий спектр возможностей для создания многопользовательских проектов. Unity сочетает в себе визуальное редактирование сцен, поддержку 2D- и 3D-графики, встроенную физику, анимационные инструменты и гибкие средства создания пользовательского интерфейса. Одним из ключевых преимуществ Unity Engine 6 является

поддержка нескольких клиентов на одном устройстве, что упрощает локальное тестирование сетевых механик без необходимости развертывания выделенного сервера.

Кроме того, Unity позволяет реализовать поведение игровых объектов с использованием языка программирования C#, что даёт разработчику широкие возможности при создании логики кооперативного взаимодействия между игроками, а также управления элементами сетевого геймплея. Благодаря интеграции с магазином ассетов и активному сообществу разработчиков, Unity предоставляет доступ к множеству готовых решений и дополнений, что значительно ускоряет процесс разработки и помогает оперативно решать возникающие технические задачи.

1.2 Visual Studio

Для разработки и отладки кода использовалась интегрированная среда разработки (IDE) Visual Studio 2022. Рабочая область данного инструмента представлена на рисунке 1.2.

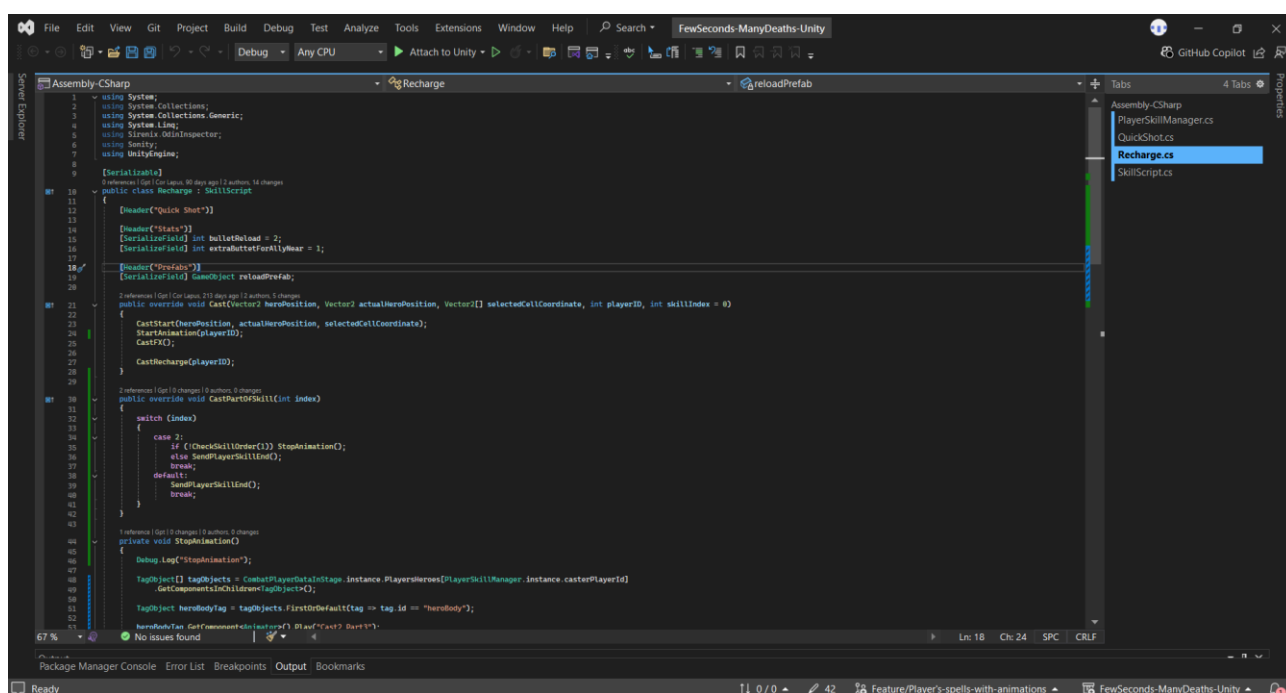


Рисунок 1.2 — Среда разработки Visual Studio Code

Visual Studio 2022 представляет собой мощную и универсальную IDE, предназначенную для разработки приложений на различных языках программирования, включая C#, C++, Python и другие. Среда предлагает широкие возможности для создания программного обеспечения, включая поддержку отладки, тестирования, анализа кода и интеграции с системами контроля версий, такими как Git. Visual Studio 2022 включает в себя множество инструментов для работы с проектами, создание пользовательских интерфейсов, а также поддержку многозадачности и параллельной разработки.

Одним из ключевых преимуществ Visual Studio 2022 является её масштабируемость и высокая производительность. Программа поддерживает работу с большими проектами и решениями, значительно улучшив оптимизацию работы с кодом благодаря улучшенной многозадачности и ускорению процессов компиляции. Встроенные инструменты для отладки и профилирования позволяют разработчику эффективно находить и устранять ошибки в коде, а также анализировать производительность приложений.

Кроме того, Visual Studio 2022 предоставляет богатую экосистему плагинов и расширений, что позволяет адаптировать IDE под конкретные нужды разработчика. Встроенная поддержка таких технологий, как Azure, Docker, и интеграция с инструментами DevOps, делает её идеальным выбором для разработки как локальных приложений, так и облачных решений. Использование Visual Studio 2022 в процессе разработки проекта позволило оптимизировать процесс кодирования и повысить эффективность командной работы.

1.3 Fork

В процессе разработки проекта для управления версиями исходного кода использовалась система контроля версий Git в сочетании с графическим клиентом Fork. Интерфейс данного инструмента представлен на рисунке 1.3.

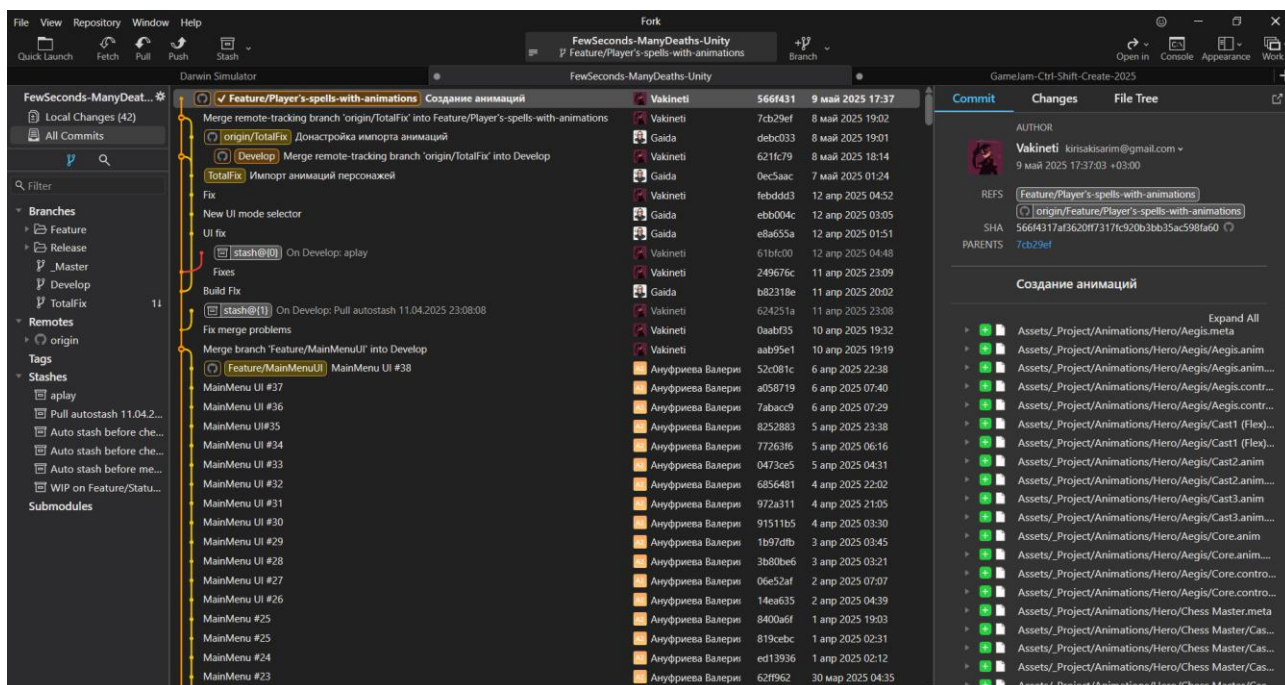


Рисунок 1.3 — Графический клиент Fork

Fork представляет собой современный и удобный графический интерфейс для работы с системой Git, предоставляющий разработчику визуальные средства управления репозиториями, ветками и коммитами. Приложение включает все необходимые функции для эффективной организации командной разработки: клонирование репозитория, создание и слияние веток, разрешение конфликтов, выполнение rebase и cherry-pick операций, а также просмотр истории изменений в наглядной древовидной структуре.

Одним из преимуществ Fork является интуитивно понятный интерфейс, позволяющий даже при минимальном опыте в Git быстро освоиться в работе с репозиториями. Программа поддерживает интеграцию с такими платформами, как GitHub, GitLab и Bitbucket, что облегчает взаимодействие между участниками команды. Кроме того, Fork позволяет выполнять операции как через графический интерфейс, так и с использованием встроенного терминала, что делает его универсальным инструментом как для начинающих, так и для опытных разработчиков.

Использование Fork в рамках проекта способствовало упрощению контроля над изменениями в коде, обеспечению стабильной работы с ветками и повышению прозрачности совместной разработки в команде.

Глава 2. Конструкторская часть

2.1 Разработка дизайн-документа

2.1.1. Общая информация о проекте

Проект *Few Seconds – Many Deaths* представляет собой кооперативную тактическую 2D-игру в жанре пошаговой стратегии, разработанную для платформы ПК с использованием Unity 6. Основная цель игры — предоставить игрокам динамичный и стратегический геймплей, сосредоточенный на командном взаимодействии и сражениях с мощным боссом на клеточном поле. Игроки (до четырёх человек) совместно планируют действия, используют уникальные навыки персонажей и адаптируются к атакам босса, чтобы выжить и победить.

Целевая аудитория включает игроков, предпочитающих тактические и кооперативные игры, такие как *Into the Breach* или *Divinity: Original Sin 2*, с акцентом на стратегическое планирование и командную работу. Игра ориентирована на пользователей, знакомых с пошаговыми механиками, но доступна для новичков благодаря интуитивному интерфейсу и визуальной обратной связи. Основной режим — кооперативный многопользовательский, с поддержкой сетевой синхронизации для обеспечения плавного взаимодействия между игроками.

Проект разрабатывается с использованием модульной архитектуры, включающей системы управления сессиями, боевыми стадиями, перемещением, навыками и действиями босса. Используемые технологии — Unity 6, Netcode for GameObjects и Unity Transport — обеспечивают стабильную работу многопользовательского режима и оптимизированную производительность. Дизайн-документ фиксирует концепцию, жанр, платформу и ключевые цели, создавая основу для последующих этапов разработки.

2.1.2. Определение основных особенности игры

Основные особенности включают:

- Пошаговая боевая система: Бой разделён на четыре стадии — предсказание (демонстрация атак босса), выбор действий игроков, выполнение действий игроков и действия босса. Эта структура, реализованная через `CombatStageManager`, обеспечивает тактическую глубину и требует координации между игроками. Игровой цикл представлен на рисунке 2.1.

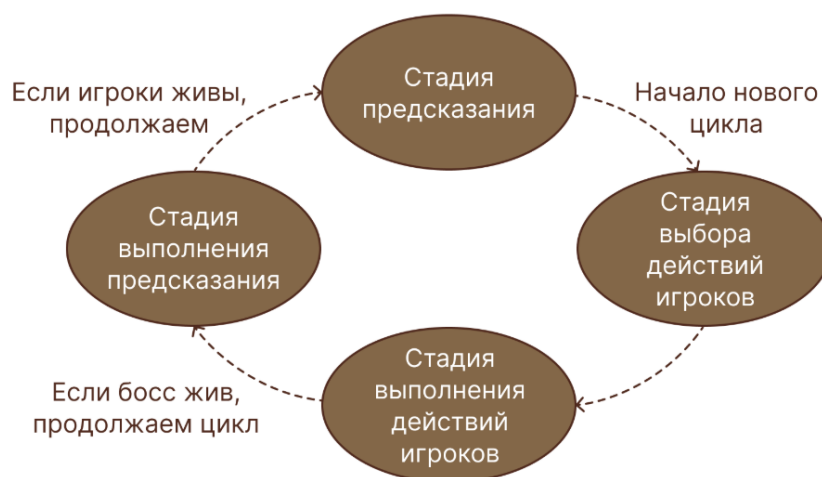


Рисунок 2.1 — Схематическое отображение основного игрового цикла

- Кооперативный режим: Поддержка до четырёх игроков в многопользовательском режиме через `Netcode for GameObjects`. Система сессий (`SessionsManager`) позволяет создавать и присоединяться к лобби, а синхронизация данных (`PlayerDataSyncService`) обеспечивает согласованность персонажей, навыков и игровых состояний.
- Гибкая система навыков: Навыки игроков, основанные на `ScriptableObject` (`SkillScript`), включают разнообразные механики, такие как нанесение урона (`QuickShot`), наложение щитов

(HorizontalSign) или лечение. Гибкость архитектуры позволяет легко добавлять новые навыки.

- Динамичные действия босса: Босс выполняет случайные комбинации атак (BossActionScript), такие как круговая атака (BerserkCircularSlice_Berserk) или рывок (Dash_Berserk), с изменением поведения в зависимости от фазы боя (ChangeAct). Призрачный спрайт (GhostBossGameObject) визуализирует атаки на стадии предсказания.
- Случайная очередность ходов: Механика, реализованная в TurnOrderManager, случайно определяет порядок действий игроков в каждом цикле боя, добавляя непредсказуемость и требуя адаптации.
- Визуализация и оптимизация: Клеточное поле (MapClass) и зоны действия визуализируются с использованием пулов объектов (SkillTilePoolManager), что минимизирует нагрузку на производительность. Интерфейс (GridVisualizer) обеспечивает чёткую обратную связь о перемещениях и навыках.

Эти особенности подчёркивают тактический и кооперативный характер игры, создавая реиграбельный и стратегический опыт, где успех зависит от командной работы и планирования.

2.1.3. Технические требования

Игра разрабатывается на движке Unity Engine версии 6000.0.23f1 с использованием языка C# и технологий Netcode for GameObjects и Unity Transport для реализации сетевого взаимодействия.

Технические требования к системе минимальны и представлены в Таблице 1.

Таблица 1 — Системные требования

Система	Минимальное требование
Операционная система	Windows 10 / 11
Процессор	Двухъядерный, от 2.5 GHz
Оперативная память	От 8 GB
Видеокарта	С поддержкой DirectX 11
Разрешение	1280×720 и выше
Свободное место на диске	Около 1 GB
Сетевое подключение	Не менее 5 Мбит/с

2.2 Описание механик

2.2.1. Общая архитектура системы

Архитектура игры построена на модульной структуре, обеспечивающей гибкость, масштабируемость и удобство поддержки. Основные подсистемы включают управление игровыми сессиями, синхронизацию данных, боевую систему, перемещение персонажей, систему навыков и действия босса. Взаимодействие между модулями осуществляется через событийную систему `GlobalEventSystem`, которая передаёт уведомления о ключевых событиях (например, `PlayerTurnStageStarted`, `BossActionEnd`). Синглтоны, такие как `SessionsManager`, `PlayerInfoData` и `CombatStageManager`, обеспечивают централизованное управление данными и состоянием игры.

- Управление сессиями: Реализовано через `SessionsManager`, который отвечает за создание и подключение к игровым лобби с использованием `SessionService` и `Unity Multiplayer Services`. Поддерживается анонимная авторизация и релейная сеть для стабильного соединения. Общая архитектура системы мультиплеера представлена на рисунке 2.2.

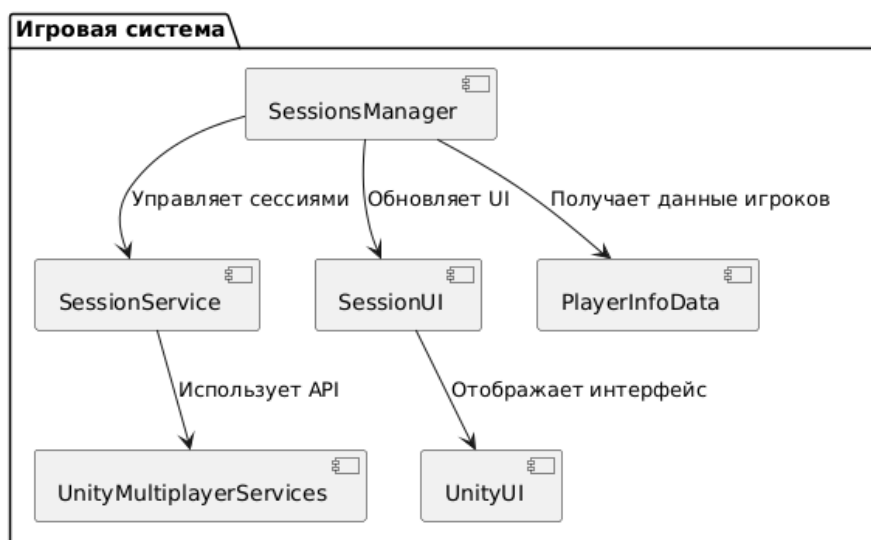


Рисунок 2.2 — Общая архитектура системы мультиплеера

- Синхронизация данных: **PlayerDataSyncService** и **PlayerInfoData** синхронизируют информацию о персонажах, цветах и навыках через RPC (**ChangeHeroRpc**, **ChangeSkillRpc**), обеспечивая согласованность в кооперативном режиме.
- Боевая система: Управляется **CombatStageManager**, который циклически переключает стадии боя (предсказание, выбор действий, выполнение действий игроков, действия босса) с использованием **NetworkVariable** для сетевой синхронизации.
- Перемещение и навыки: **PlayerMovementController** и **PlayerSkillManager** реализуют тактическое перемещение по клеточному полю и применение навыков, интегрированные с **MapClass** и **SkillScript** на основе **ScriptableObject**.
- Действия босса: **BossManager** управляет выбором и выполнением комбинаций атак (**ChoiceCombo**, **CastCombo**), синхронизируемых через **CastActionRpc** и визуализируемых с помощью **GhostBossGameObject**.

Модульная архитектура позволяет легко добавлять новые механики, такие как дополнительные навыки или фазы босса, а использование **Netcode for GameObjects** обеспечивает стабильную сетевую синхронизацию, критически важную для кооперативного геймплея.

2.2.2. Проектирование игровых механик

Игровые механики спроектированы для создания динамичного и тактического геймплея, требующего от игроков координации и стратегического планирования. Основные элементы включают перемещение по клеточному полю, систему навыков игроков, действия босса и визуализацию игровых действий.

- Пошаговая боевая система: Состоит из четырёх стадий (предсказание, выбор действий, выполнение действий игроков, действия босса). Игроки одновременно выбирают действия в ограниченное время (`maxTurnTime`), а их выполнение происходит в случайном порядке, заданном `TurnOrderManager`. Это стимулирует адаптацию и командное планирование.
- Перемещение по клеточному полю: Реализовано через `GameplayTilemap` с использованием алгоритма поиска пути (`GridPathfinding`). Перемещение расходует энергию: 2 единицы за горизонтальное/вертикальное движение, 3 единицы за диагональное. Клетки, занятые союзниками или боссом, проходимы, но остановка на них запрещена (`MapClass.IsPlayable`). Выбор клетки перемещения указан на рисунке 2.3.

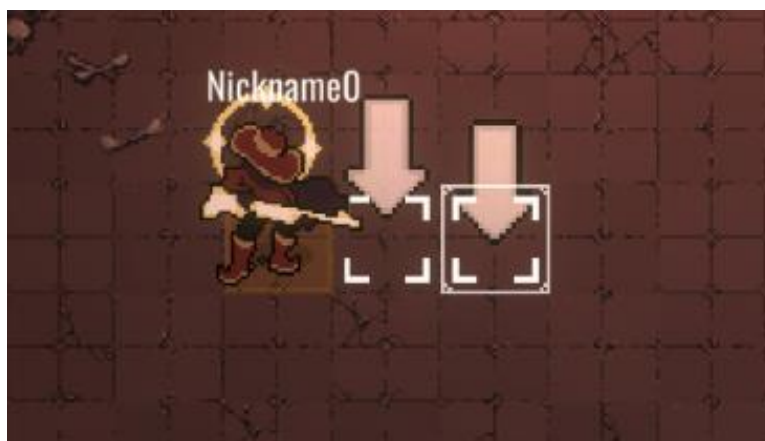


Рисунок 2.3 — Выбор клетки перемещения

- Система энергии: Энергия (`EnergyPerTurn`) восполняется в начале каждого цикла на стадии предсказания (`RestoreEnergy`). Расход энергии зависит от

типа действия: перемещение или применение навыков (задаётся EnergyCost в SkillScript).

- Навыки персонажей: Каждый персонаж имеет уникальные способности, реализованные через SkillScript. Например, HorizontalSign накладывает щит на союзников в горизонтальной линии, а QuickShot наносит урон с учётом боеприпасов и бонусов за позиционирование. Навыки требуют координации для создания синергетических эффектов, таких как защита и атака.
- Действия босса: Реализованы через BossActionScript с использованием случайного выбора комбинаций атак (ChoiceCombo). Например, BerserkCircularSlice_Berserk создаёт круговую зону поражения, а Dash_Berserk выполняет рывок с уроном по пути. Параметры атак изменяются в зависимости от фазы боя (act).
- Визуализация: Зоны действия навыков и атак босса отображаются через SkillVisualizer и BossTileManager, используя пулы объектов (SkillTilePoolManager) для оптимизации. Призрачный спрайт (GhostBossGameObject) показывает действия босса на стадии предсказания.

2.2.3. Алгоритмы и технические решения

Для реализации игровых механик применены следующие алгоритмические и технические подходы, обеспечивающие производительность и стабильность:

- Алгоритм поиска пути: GridPathfinding строит маршруты перемещения, учитывая проходимость клеток (MapClass.IsPlayable) и препятствия. Метод CreatePathRoute создаёт временный путь, подтверждаемый игроком через ApproveThePath, с коррекцией для избежания коллизий (CorrectingPath).
- Случайная очередность ходов: TurnOrderManager использует алгоритм случайного перемешивания (RandomiseList с System.Random) для

определения порядка действий игроков. Порядок синхронизируется через `SetNewTurnOrderRpc`, обновляя интерфейс (`playersIconsList`).

- Система навыков: Навыки реализованы через `ScriptableObject (SkillScript)`, где параметры (`EnergyCost`, `SkillCooldown`, `TargetCount`) задаются в редакторе Unity. `PlayerSkillManager` проверяет условия применения (энергия, перезарядка) и активирует навыки через корутины (`Cast`, `ShotMovement`), синхронизируемые с помощью `CastActionRpc`.
- Синхронизация боя: `CombatStageManager` использует `NetworkVariable<int>` для хранения текущей стадии боя (`currentStageIndexNet`), обеспечивая согласованность через RPC (`NextStage`). События `GlobalEventSystem` (`PlayerTurnStageEnded`, `BossTurnStageStarted`) уведомляют о смене стадий.
- Действия босса: Алгоритм выбора комбинаций атак (`ChoiceCombo`) использует случайный выбор из `ComboAttackList` с учётом фазы боя. `GetTargetPointsForActions` задаёт цели, синхронизируемые через `GetTargetPointsForActionsRpc`.
- Объектный пулинг: `SkillTilePoolManager` управляет пулами объектов (`attackTilePool`, `shieldTilePool`) для визуализации зон действия и эффектов, минимизируя создание/удаление объектов (`SpawnSkillObjects`, `HideSkillObjects`).
- Сетевая синхронизация: Авторитарная модель использует хост для управления критическими событиями (перемещение, навыки, действия босса), а клиенты синхронизируются через RPC (`ChangePlayerCoordinatesRpc`, `CastActionRpc`) и `NetworkVariable`. Это минимизирует задержки и рассинхронизацию.

Эти решения обеспечили гибкую и оптимизированную архитектуру, поддерживающую кооперативный геймплей. Модульная структура и событийная система упрощают добавление новых механик, таких как дополнительные навыки, классы персонажей или типы боссов, в будущем.

Глава 3. Практическая часть

3.1. Сетевое взаимодействие

В проекте Few Seconds – Many Deaths сетевое взаимодействие реализовано с использованием Unity 6, Netcode for GameObjects и Unity Transport. Архитектура построена на модульной системе, включающей управление сессиями, синхронизацию данных игроков и игровых состояний, а также интерфейс для взаимодействия с лобби. Это обеспечивает стабильную работу кооперативного режима, позволяя игрокам совместно проходить уровни и координировать действия. Модули взаимодействуют через событийную систему GlobalEventSystem и синглтоны, такие как SessionsManager и PlayerInfoData, что упрощает масштабируемость и тестирование.

3.1.1. Управление игровыми сессиями

Управление игровыми сессиями реализовано через класс SessionsManager, который отвечает за создание, поиск и подключение к игровым лобби, обеспечивая кооперативное взаимодействие игроков. В методе Awake класса SessionsManager инициализируется пул объектов (sessionPool) для динамического отображения списка сессий в интерфейсе, а также создается экземпляр SessionService, взаимодействующий с Unity Multiplayer Services. В методе Start выполняется анонимная авторизация через AuthenticationService.Instance.SignInAnonymouslyAsync, что позволяет игрокам подключаться без необходимости создания учетной записи.

Создание сессии осуществляется методом StartSessionAsHost, где задаются параметры лобби: режим игры (sessionMod), приватность (IsPrivate), максимальное количество игроков (maxPlayers) и пароль (Password). Метод использует SessionService.CreateSessionAsync для создания сессии с поддержкой релейной сети (WithRelayNetwork). После успешного создания сессии

вызывается метод `InitialiseSession`, который сохраняет данные сессии в `LobbySaver.instance.currentSession`, активирует игровой интерфейс через `SessionUI.SetUIState(UIState.InLobby)` и обновляет информацию о лобби (код сессии, пароль) с помощью `SessionUI.UpdateLobbyInfo`.

Подключение к существующим сессиям возможно двумя способами: по уникальному коду через `JoinSessionByCodeAsync` или по идентификатору сессии через `JoinSessionByIdAsync`. В первом случае игрок вводит код в поле `joinCodeInputField`, и метод `JoinSessionByCode` обрабатывает подключение. Во втором случае используется метод `JoinSession`, который проверяет наличие пароля у сессии (`HasPassword`) и, при необходимости, активирует интерфейс ввода пароля через `SessionUI.ShowPasswordPrompt`. Если пароль введен корректно, метод `JoinSessionWithPassword` вызывает `TryJoinSession` для подключения.

Обновление списка доступных сессий реализуется через корутину `UpdateSessionListCoroutine`, которая с интервалом `sessionUpdateInterval` запрашивает данные через `SessionService.GetSessionsList`. Полученные сессии отображаются в интерфейсе с помощью `SessionUI.UpdateSessionList`, где для каждой сессии указываются название, статус приватности и количество подключенных игроков. Интерфейс поддерживает функционал копирования кода и пароля (`CopyText`) и отмены ввода пароля (`CancelPasswordPrompt`), обеспечивая удобство взаимодействия.

3.1.2. Синхронизация данных игроков

Синхронизация данных игроков в проекте реализована через класс `PlayerDataSyncService`, который обеспечивает передачу информации о персонажах, их цветах и выбранных навыках между клиентами в кооперативном режиме. Этот процесс гарантирует, что все игроки имеют актуальные данные о состоянии друг друга, что критически важно для согласованного игрового процесса.

Основной компонент, `PlayerDataSyncService`, работает в связке с `PlayerInfoData`, который выступает синглтоном для хранения данных игроков, включая списки цветов (`ColorList`), персонажей (`HeroDataList`) и выбранных навыков (`SkillChoiceList`). При подключении нового клиента метод `OnNewClientConnected` запускает корутину `WaitForPlayerInfoDataInitialization`, которая ожидает инициализации `PlayerInfoData` и вызывает `SendDataToNewClient`. Этот метод отправляет данные о персонаже (`ChangeHeroNewClientRpc`), цвете (`ChangeColorNewClientRpc`) и навыках через `PlayerDataSkillManager.SyncSkillsForClient` с использованием RPC и `ClientRpcParams` для таргетинга конкретного клиента.

Листинг 3.1 — метод синхронизации героев между игроками

[`ClientRpc`]

```
private void ChangeHeroNewClientRpc(int ID, int heroDataID, ClientRpcParams
clientRpcParams = default)
{
    playerDataService.SetHero(ID, heroDataID);
    GlobalEventSystem.SendHeroChanged();
}
```

Класс `ChangePlayersListNetwork` обрабатывает изменения персонажей и навыков. Метод `ChangeHero` получает идентификатор персонажа через `GetHeroDataID` и передает его через `ChangeHeroRpc`, синхронизируя выбор персонажа на всех клиентах. Аналогично, метод `ChangeSkill` обновляет выбранный навык через `ChangeSkillRpc`, используя индексы навыков и их вариаций. Эти RPC-вызовы инициируют события `GlobalEventSystem` (`SendHeroChanged`, `SendSkillChanged`), которые уведомляют другие системы об обновлениях.

Данные игроков хранятся в структуре `PlayerInfoData`, где `PlayerCount` отслеживает количество подключенных игроков, а `PlayerIDThisPlayer`

идентифицирует текущего игрока. События `PlayerLobbyUpdate` и `PlayerDataChanged` обеспечивают обновление данных через метод `UpdatePlayerNickList`, который синхронизирует никнеймы и другие параметры. Для предотвращения дублирования персонажей используется метод `HasDuplicates`, проверяющий уникальность выбранных героев перед стартом игры.

Синхронизация выполняется в авторитарной модели, где хост управляет критическими данными, а клиенты получают обновления через RPC. Это минимизирует задержки и обеспечивает стабильность, особенно при динамическом подключении или отключении игроков, обрабатываемых через `ConnectPlayer` и `DisconnectPlayer` в `PlayerInfoData`. Такой подход гарантирует целостность игровых данных и их согласованность на всех клиентах, что является основой для кооперативного взаимодействия в игре.

3.1.3. Сетевая синхронизация игровых состояний

Сетевая синхронизация игровых состояний в *Few Seconds – Many Deaths* обеспечивает согласованность игрового процесса между всеми клиентами, включая позиции персонажей, стадии боя и действия игроков. Основной задачей является поддержание целостности игровых данных в кооперативном режиме, что достигается с использованием технологий `Netcode for GameObjects` и `Unity Transport`. Синхронизация реализована через комбинацию компонентов `ClientNetworkTransform`, `NetworkVariable` и RPC (Remote Procedure Calls), что позволяет передавать данные о перемещениях, навыках и очередности ходов между хостом и клиентами.

Перемещение персонажей по клеточному полю осуществляется с использованием класса `MapClass`, который строит пути на основе алгоритма `Pathfinding`. Данные о перемещении передаются хостом через RPC, вызываемые в `PlayerMovementController`. Метод отправляет ID игрока и список клеток пути, после чего клиенты выполняют соответствующее перемещение,

синхронизированное через ClientNetworkTransform. Это гарантирует, что позиции персонажей одинаковы на всех клиентах, исключая рассинхронизацию.

Синхронизация стадий боя (предсказание, выбор действий, выполнение действий игроков, выполнение предсказания босса) реализована через CombatStageManager и NetworkVariable. NetworkVariable хранит текущую стадию боя, автоматически синхронизируя её между хостом и клиентами. Переход между стадиями инициируется хостом, который уведомляет клиентов через RPC, обеспечивая одновременное обновление игрового состояния. Например, в стадии выбора действий игроки используют PlayerSkillManager для передачи информации о выбранных навыках (ID игрока, индекс навыка, координаты каста) через метод CastActionRpc. Эти данные отправляются хосту, который координирует их выполнение и рассылает результаты клиентам.

Очередность ходов управляется классом TurnOrderManager, который случайным образом определяет порядок действий игроков в начале каждого цикла. Метод RandomiseTurnOrder использует алгоритм перемешивания (System.Random) для генерации новой последовательности, которая синхронизируется через RPC SetNewTurnOrderRpc. Этот метод обновляет список TurnPriority в CombatPlayerDataInStage и передает иконки персонажей для отображения порядка ходов на всех клиентах. Случайная очередность добавляет элемент непредсказуемости, требуя от игроков адаптации к изменяющимся условиям.

Для оптимизации производительности используется объектный пулинг для визуальных эффектов, таких как индикаторы зон поражения, что снижает нагрузку на систему при частых обновлениях. Событийная система GlobalEventSystem уведомляет клиентов о ключевых изменениях, таких как завершение действия (SendPlayerEndResultTurn) или обновление состояния боя (SendBossActionEnd). Это обеспечивает согласованность визуальных и игровых данных.

Таким образом, сетевая синхронизация игровых состояний в проекте реализована через эффективное сочетание ClientNetworkTransform,

NetworkVariable и RPC, обеспечивая стабильный и согласованный кооперативный геймплей. Тестирование показало, что система успешно справляется с синхронизацией данных при подключении четырёх игроков, минимизируя задержки и обеспечивая плавный игровой процесс.

3.1.4. Интерфейс сетевого взаимодействия

Реализация пользовательского интерфейса для сетевого взаимодействия осуществляется через класс SessionUI, который управляет визуальными и функциональными аспектами взаимодействия с игровыми сессиями. Этот компонент отвечает за отображение меню поиска сессий, информации о текущем лобби и обработку ввода данных, таких как пароль для защищенных сессий.

Класс SessionUI интегрирован с SessionsManager и использует перечисление UIState (MainMenu, SessionSearch, InLobby) для управления состояниями интерфейса. В состоянии MainMenu отображается главное меню, в SessionSearch — список доступных сессий, а в InLobby — информация о текущем лобби, включая код сессии и пароль (если установлен). Метод SetUIState переключает видимость соответствующих панелей (MainMenu, SessionSearchMenu, InLobbyMenu) в зависимости от текущего состояния.

Метод UpdateSessionList обновляет список доступных сессий, получаемых через SessionService.GetSessionsList. Для каждой сессии создается элемент интерфейса из пула объектов (sessionPool), отображающий название сессии, статус приватности (HasPassword) и количество игроков (MaxPlayers - AvailableSlots). Кнопка присоединения (joinButton) вызывает метод OnJoinSession, который передает данные сессии в SessionsManager.JoinSession.

Для защищенных сессий реализован механизм ввода пароля. Метод ShowPasswordPrompt активирует панель ввода пароля (passwordPromptPanel), а HidePasswordPrompt скрывает ее после успешного подключения или отмены. Кнопка подтверждения (confirmPasswordButton) вызывает JoinSessionWithPassword из SessionsManager, передавая введенный пароль. При

неверном пароле метод `ShowErrorMessage` отображает сообщение об ошибке через `errorMessageText`.

Интерфейс также поддерживает функции копирования кода сессии и пароля через метод `CopyText`, связанный с кнопками `copyJoinCodeButton` и `copyPasswordButton`. Кнопка отмены (`cancelPasswordButton`) вызывает `CancelPasswordPrompt`, сбрасывая ввод и скрывая панель пароля. Метод `UpdateLobbyInfo` отображает информацию о текущей сессии (код, пароль) только для хоста, активируя панель `LobbyInfoPanel` и, при необходимости, `passwordPanel`. UI лобби показан на рисунке 3.1.

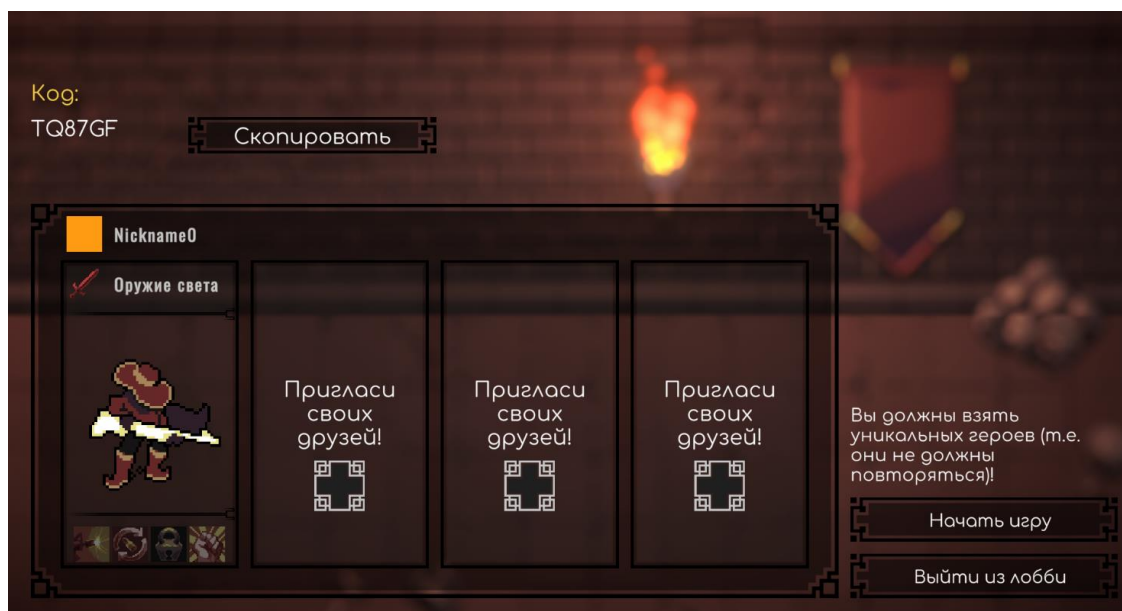


Рисунок 3.1 — UI лобби

Данная структура интерфейса обеспечивает интуитивное управление сетевыми функциями, упрощая процесс создания и подключения к сессиям, а также обеспечивая обратную связь для игроков через визуальные элементы и сообщения об ошибках.

3.2 Пошаговая система боя

Пошаговая система боя построена на четырех повторяющихся стадиях: предсказание, выбор действий игроков, выполнение действий игроков и

выполнение действий босса. Управление стадиями осуществляется через класс `CombatStageManager`, а синхронизация между клиентами обеспечивается с использованием `Netcode for GameObjects`. Механика включает случайную очередность ходов, управление энергией и интерфейс для взаимодействия игроков, что способствует стратегическому планированию и координации.

3.2.1. Архитектура боевой системы

Архитектура пошаговой боевой системы в проекте *Few Seconds – Many Deaths* реализована через класс `CombatStageManager`, который выступает центральным компонентом, управляющим всеми стадиями боя. Система построена на модульной структуре, включающей четыре стадии: предсказание (`PredictionStage`), выбор действий игроков (`PlayerTurnStage`), выполнение действий игроков (`ResultStage`) и действия босса (`BossTurnStage`). Эти стадии хранятся в списке `_stages` и последовательно активируются, обеспечивая циклический игровой процесс.

Класс `CombatStageManager` инициализируется в методе `OnNetworkSpawn`, где создаются экземпляры всех стадий и вызывается метод `Initialize` для их настройки. Переходы между стадиями управляются методом `TransitionToNextStage`, который использует `NetworkVariable<int> currentStageIndexNet` для синхронизации текущей стадии между сервером и клиентами через `Netcode for GameObjects`. На сервере метод `NextStage` активирует следующую стадию, а клиенты обновляют состояние в методе `Update`, отслеживая изменения `currentStageIndexNet`. Класс-диаграмма боевой системы указана на рисунке 3.2.

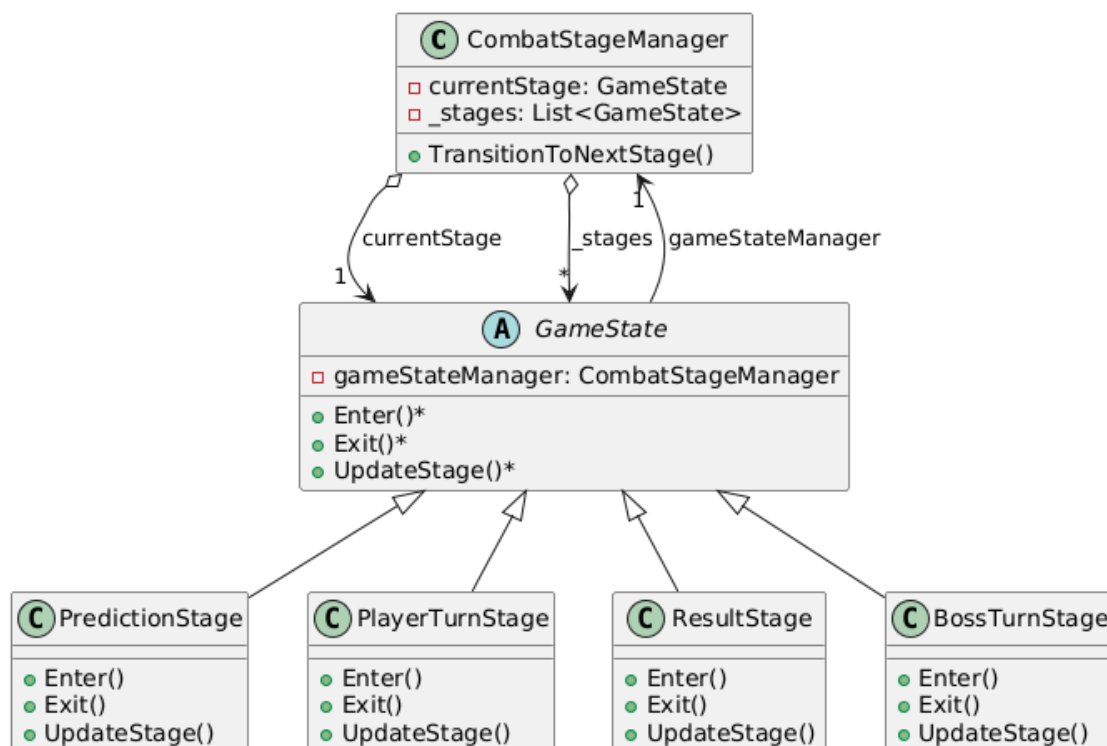


Рисунок 3.2 — Класс-диаграмма боевой системы

Событийная система `GlobalEventSystem` играет ключевую роль в управлении боем. Событие `PlayerDataUpdated` запускает бой через `StartBattle`, вызывая первую стадию. События `AllPlayersDied` и `BossDied` завершают бой, активируя соответствующие меню (`LoseMenu` или `WinMenu`) и устанавливая флаг `isCombatRunning` в `false`. Каждая стадия наследуется от абстрактного класса `GameState`, реализующего методы `Enter`, `Exit` и `UpdateStage` для управления входом, выходом и обновлением состояния.

Синхронизация между клиентами обеспечивается авторитарной моделью, где сервер управляет критическими событиями, а клиенты получают обновления через `RPC` и сетевые переменные. Флаг `isCombatRunning` контролирует активность боя, предотвращая выполнение стадий после его завершения. Интерфейс, включающий таймер (`timerText`), текст подтверждения (`confirmationText`) и кнопку (`confirmationButton`), интегрирован с системой для отображения текущего состояния боя.

Такая архитектура обеспечивает гибкость, позволяя легко добавлять новые стадии или модифицировать существующие, а также гарантирует стабильную

сетевую синхронизацию, что критично для кооперативного геймплея. Модульность и использование событийной системы упрощают тестирование и дальнейшее развитие боевой механики.

3.2.2. Стадия предсказания

Стадия предсказания, реализованная в классе `PredictionStage`, является первой фазой пошаговой боевой системы в проекте `Few Seconds – Many Deaths`. На этой стадии босс демонстрирует свои будущие атаки, не нанося урона и не накладывая эффекты, что дает игрокам возможность спланировать свои действия. Основная цель стадии — предоставить информацию о предстоящих действиях босса, способствуя стратегическому взаимодействию команды.

При входе в стадию метод `Enter` инициирует выбор комбо босса через `BossManager.ChoiceCombo`, который выполняется на сервере. Это гарантирует, что все клиенты получают согласованные данные о действиях босса. Одновременно восстанавливается энергия игроков с помощью метода `RestoreEnergy`, который добавляет фиксированное количество энергии (`EnergyPerTurn`) к текущему значению каждого игрока, не превышая максимального лимита (`MaxEnergy`). Энергия синхронизируется между клиентами через RPC-вызов `ChangePlayerEnergyRpc`. Учет эффектов, изменяющих энергию, реализован в методе `EnergyUpdate`, который проверяет наличие модификаторов через `EffectsManager`.

Метод `ShieldsRemove` сбрасывает щиты босса и всех игроков, устанавливая значения `CurrentShield` в ноль. Это действие синхронизируется через событие `GlobalEventSystem.SendPlayerShieldChanged`, обновляющее интерфейс на всех клиентах. Событие `BossEndCombo`, вызываемое после завершения анимации комбо босса, инициирует переход к следующей стадии через `CombatStageManager.TransitionToNextStage`.

Визуальная обратная связь обеспечивается через `FeelFeedbacksManager.SetActivePredictionEffect`, активирующий эффект,

указывающий на стадию предсказания. События `GlobalEventSystem` (`SendPredictionStageStarted`, `SendPredictionStageEnded`) уведомляют другие компоненты о начале и завершении стадии, обеспечивая корректное обновление интерфейса и игровой логики. Сетевая синхронизация, реализованная через вызовы на сервере и RPC, гарантирует, что все игроки видят одинаковую последовательность действий босса и обновленные параметры энергии и щитов.

3.2.3. Стадия выбора действий игроков

Стадия выбора действий игроков в проекте `Few Seconds – Many Deaths` реализована через класс `PlayerTurnStage`, который управляет процессом принятия решений игроками в рамках ограниченного времени. Эта стадия является ключевой для кооперативного геймплея, позволяя каждому игроку одновременно выбирать перемещения или навыки, что требует стратегической координации. Система интегрирована с сетевой моделью `Netcode for GameObjects`, обеспечивая синхронизацию действий между клиентами.

При входе в стадию метод `Enter` инициализирует процесс, активируя пользовательский интерфейс через `EnablePlayerTurnUI`. На сервере запускается корутина `Timer`, которая отслеживает время, доступное для выбора (`maxTurnTime`), используя `NetworkVariable<float> remainingTime` для синхронизации. Оставшееся время отображается в элементе интерфейса `timerText`, обновляемом через `UpdateTimerUIRpc`. Интерфейс, управляемый через `GUIDataBase`, включает кнопки выбора навыков и кнопку подтверждения (`confirmationButton`), которая становится активной для живых игроков (`aliveStatus` из `CombatPlayerDataInStage`).

Игроки подтверждают свои действия, нажимая кнопку подтверждения или используя ввод через `InputActions.Combat.EndPlayerTurn`, что вызывает метод `ConfirmEndTurn`. Этот метод отключает интерфейс выбора (`EndTurnButtonActiveChange`) и отправляет серверу запрос через `ConfirmEndTurnRpc`, увеличивая счетчик подтверждений (`playersConfirmed`,

синхронизируемый через `NetworkVariable<int>`). Текущее количество подтверждений отображается в `confirmationText` в формате "текущее/всего живых игроков", обновляемом через `UpdateConfirmationUI`.

Если все живые игроки подтверждают действия (`playersConfirmed.Value >= CountOfAlivePlayers`) или время истекает (`remainingTime.Value <= 0`), сервер вызывает `StartEndingTurn`, который отключает интерфейс, деактивирует ввод (`inputActions.Disable`) и отправляет событие `PlayerTurnEnding` через `GlobalEventSystem`. Это завершает стадию, иницируя переход к следующей через `TransitionToNextStage` из `CombatStageManager`. Для клиентов, не завершивших выбор, действия автоматически подтверждаются по истечении времени.

Стадия завершается событием `PlayerTurnStageEnded`, отправляемым через `GlobalEventSystem`, а интерфейс отключается через `EnablePlayerTurnUI(false)`. Сетевая синхронизация обеспечивает согласованность таймера и подтверждений, а интеграция с `GUIDataBase` гарантирует интуитивное взаимодействие, позволяя игрокам сосредоточиться на тактическом планировании.

3.2.4. Стадия выполнения действий игроков

Стадия выполнения действий игроков реализована через класс `ResultStage`, который отвечает за последовательное выполнение перемещений и навыков, выбранных игроками на предыдущей стадии. Эта стадия обеспечивает синхронизированное воспроизведение действий в порядке, определенном `TurnOrderManager`, с учетом сетевой синхронизации для всех клиентов. Основной задачей является корректное выполнение игровых действий с визуальной обратной связью, поддерживая динамику и последовательность кооперативного боя.

При входе в стадию метод `Enter` иницирует процесс выполнения действий, вызывая `SendStartResultStageForPlayerRpc` с задержкой, заданной

параметром `TimeBeforeFirstPlayerMovement`. Это обеспечивает плавное начало стадии, давая клиентам время на обновление интерфейса. Событие `GlobalEventSystem.SendResultStageStarted` уведомляет всех клиентов о начале стадии. На сервере счетчики `PlayerEndMovind` и `PlayerEndResultTurn`, реализованные как `NetworkVariable`, обнуляются для отслеживания завершенных перемещений и действий.

Процесс выполнения действий делится на два этапа: перемещение и применение навыков. Перемещение персонажей по клеточному полю начинается с вызова `SendStartResultStageForPlayerRpc` для игрока с текущим приоритетом (`_orderInTurnPriority`). После завершения перемещения событие `PlayerEndMoving` увеличивает `PlayerEndMovind` через `ConfirmPlayerEndMovingRpc`, а следующий игрок получает команду на выполнение с задержкой `TimeBetweenPlayerMovement`. Когда все перемещения завершены (`PlayerEndMovind.Value >= playerCount`), сервер вызывает `SendStartCastPlayer` с задержкой `TimeBeforeFirstPlayerCast`, иницируя этап применения навыков.

Применение навыков происходит аналогично: событие `PlayerEndResultTurn` увеличивает `PlayerEndResultTurn` через `ConfirmPlayerEndResultTurnRpc`, а `SendAllPlayersEndMovingRpc` уведомляет клиентов о начале выполнения навыков для следующего игрока. Задержки между действиями обеспечивают визуальную читаемость, позволяя игрокам отслеживать ход боя. Интерфейс обновляется через события `GlobalEventSystem`, такие как `SendStartResultStageForPlayer` и `SendStartCastPlayer`, отображая анимации перемещений и эффекты навыков.

Стадия завершается, когда все игроки выполнили свои действия (`PlayerEndResultTurn.Value >= playerCount`), после чего вызывается `EndTurn`, переводящий бой в стадию действий босса. Метод `Exit` отправляет событие `GlobalEventSystem.SendResultStageEnded`, уведомляя клиентов о завершении стадии. Сетевая синхронизация, реализованная через `RPC` и `NetworkVariable`,

гарантирует согласованность порядка и результатов действий между сервером и клиентами, минимизируя рассинхронизацию.

3.2.5. Стадия действий босса

Стадия действий босса в проекте Few Seconds – Many Deaths реализована через класс `BossTurnStage` и является заключительным этапом цикла пошаговой боевой системы. На этой стадии босс выполняет атаки, выбранные на стадии предсказания, нанося урон игрокам и применяя эффекты, что делает её ключевой для динамики боя. Реализация обеспечивает синхронизацию действий между клиентами с использованием `Netcode for GameObjects` и событийной системы `GlobalEventSystem`.

Класс `BossTurnStage` инициализируется в методе `Initialize`, где устанавливается связь с `CombatStageManager` и подписка на событие `BossEndCombo`, которое инициирует завершение стадии. Метод `Enter` запускает выполнение комбо босса через вызов `BossManager.CastCombo`, активируемый на сервере. Это действие сопровождается событием `SendBossTurnStageStarted`, уведомляющим клиентов о начале стадии. Атаки босса, такие как "Круговая атака" или "Энергия меча", реализуются с учетом их параметров (урон, радиус, эффекты), которые могут изменяться в зависимости от фазы боя.

Обновление стадии осуществляется в методе `UpdateStage`, где сервер отслеживает состояние боя, хотя в текущей реализации основная логика сосредоточена в `BossManager`. По завершении комбо босса событие `BossEndCombo` вызывает метод `EndingTurn`, который с задержкой (`TimeBetweenRounds`, установленной в 1 секунду) инициирует переход к следующей стадии через `CombatStageManager.TransitionToNextStage`. Задержка обеспечивает плавность игрового процесса, давая игрокам время воспринять результаты атак.

Выход из стадии происходит через метод `Exit`, который отправляет событие `SendBossTurnStageEnded`, уведомляющее клиентов о завершении.

Сетевая синхронизация обеспечивается тем, что основные действия (выбор и выполнение комбо) выполняются на сервере, а клиенты получают обновления через события `GlobalEventSystem`. Это гарантирует согласованность отображения атак и эффектов у всех игроков.

Таким образом, стадия действий босса завершает цикл боя, реализуя атаки с учетом их визуальных и игровых эффектов. Интеграция с сетевой системой и событийным управлением обеспечивает стабильность и динамичность, подчеркивая тактическую важность предшествующих стадий для противодействия боссу.

3.2.6. Управление очередностью ходов

Управление очередностью ходов в проекте реализовано через класс `TurnOrderManager`, который обеспечивает случайное определение порядка действий игроков в каждом боевом цикле. Эта механика добавляет элемент непредсказуемости, требуя от игроков адаптации к динамично меняющимся условиям боя, что усиливает стратегическую составляющую кооперативного геймплея.

Класс `TurnOrderManager` инициализируется в методе `Awake`, где он подписывается на событие `GlobalEventSystem.PredictionStageStarted`, запускающее пересчет порядка ходов в начале стадии предсказания. В методе `Start` иконки персонажей (`playersIconsList`) настраиваются в соответствии с данными из `PlayerInfoData`, отображая спрайты героев (`HeroIcon`) для активных игроков, а неиспользуемые иконки скрываются.

Основная логика определения порядка реализована в методе `RandomiseTurnOrder`, который выполняется только на хосте (`NetworkManager.Singleton.IsHost`). Метод получает текущий список приоритетов (`TurnPriority`) из `CombatPlayerDataInStage` и применяет алгоритм случайного перемешивания (`RandomiseList`). Алгоритм использует `System.Random` для обмена элементов списка, обеспечивая равновероятное

распределение порядка. Например, для четырех игроков создается случайная перестановка их ID (0, 1, 2, 3), такая как [2, 0, 3, 1], определяющая последовательность ходов.

Листинг 3.2 — метод случайного перемешивания

```
private void RandomiseList(List<int> list)
{
    System.Random rng = new System.Random();
    int elementIndex = PlayerInfoData.instance.PlayerCount;
    while (elementIndex > 1)
    {
        elementIndex--;
        int randomElementIndex = rng.Next(elementIndex + 1);
        (list[elementIndex],          list[randomElementIndex])          =
(list[randomElementIndex], list[elementIndex]);
    }
}
```

Синхронизация порядка между клиентами осуществляется через метод `SetNewTurnOrderRpc`, вызываемый с помощью RPC (`SendTo.ClientsAndHost`). Этот метод передает массив нового порядка (`newTurnOrder`) всем клиентам, обновляя `TurnPriority` в `CombatPlayerDataInStage`. Одновременно обновляется интерфейс: для каждой позиции в `playersIconsList` устанавливается соответствующий спрайт героя из `PlayerInfoData.HeroDataList`, что визуально отражает текущий порядок ходов.

Такая реализация обеспечивает корректную синхронизацию очередности в сетевой игре, минимизируя расхождения между клиентами. Случайный порядок, обновляемый в начале каждого цикла, стимулирует игроков координировать действия, учитывая возможные изменения в последовательности, и повышает реиграбельность боевых сценариев.

3.3 Перемещение и навыки персонажей

Эти механики обеспечивают тактическое взаимодействие игроков с игровым полем и противником, поддерживая кооперативный геймплей. Перемещение реализовано через алгоритм поиска пути и управление энергией, а система навыков использует ScriptableObject для гибкой настройки. Визуализация игрового поля, путей и зон действия навыков осуществляется через пулы объектов и динамическое обновление интерфейса. Сетевая синхронизация обеспечивается с помощью Netcode for GameObjects, гарантируя согласованность действий между клиентами.

3.3.1 Система перемещения персонажей

Система перемещения персонажей в Few Seconds – Many Deaths реализована через класс PlayerMovementController, обеспечивающий тактическое перемещение по клеточному полю в пошаговом режиме. Механика позволяет игрокам планировать маршруты, учитывая затраты энергии и препятствия, с полной сетевой синхронизацией для кооперативного геймплея. Перемещение интегрировано с системой боя, активируясь на стадии выбора действий (PlayerTurnStage), и визуализируется через PathVisualizer.

Класс PlayerMovementController инициализируется в методе Awake, устанавливая синглтон (instance), и в OnNetworkSpawn, где подключаются события GlobalEventSystem (PlayerTurnStageStarted, PlayerTurnEnding, StartResultStageForPlayer) и действия ввода через InputActions (SelectTile, CancelAction). Перемещение начинается в методе StartDefineMovement, активирующем ввод для живых и не оглушенных игроков (isAlive, !isStunned) и сохраняющем начальную позицию (LastPosition).

Основной метод AddNewPointsToList обрабатывает выбор клетки по координатам мыши, преобразуя их в позицию на поле (GameplayTilemap.WorldToCell). Проверяется доступность клетки

(MapClass.IsPlayable) и строится маршрут с помощью CreatePathRoute, использующего алгоритм поиска пути (GridPathfinding). Для каждой точки пути метод DefineMovement определяет тип перемещения: горизонтальное/вертикальное (HorVer, 2 энергии), диагональное (Diagonal, 3 энергии), неподходящее (TooFar) или отсутствие движения (None). Если энергии достаточно, точка добавляется в MovementList, обновляется LastPosition, а энергия уменьшается через ChangePlayerEnergyRpc. Звуковые эффекты воспроизводятся через FXManager.PlayMovementSFX или PlayLackOfEnergySFX при нехватке энергии. Событие SendEnergyUsedOnMovement уведомляет о затратах.

Листинг 3.3 — метод определения типа перемещения

```
private void DefineMovement(Vector2 PlayerCoor, Vector2 targetPoint)
{
    if (PlayerCoor.x == targetPoint.x && PlayerCoor.y == targetPoint.y)
        MovementIndex = Movement.None;
    else if (Math.Abs(PlayerCoor.x - targetPoint.x) > 1 || Math.Abs(PlayerCoor.y -
        targetPoint.y) > 1) MovementIndex = Movement.TooFar;
    else if (PlayerCoor.x == targetPoint.x || PlayerCoor.y == targetPoint.y)
        MovementIndex = Movement.HorVer;
    else MovementIndex = Movement.Diagonal;
}
```

Отмена перемещения реализована в RemoveLastPointsInList, удаляющем точки до последнего чекпоинта (MovementListCheckPoints) и восстанавливающим энергию через ChangePlayerEnergyRpc. События SendEnergyRestored и SendPathChanged обновляют интерфейс, а FXManager.PlayActionCanceledSFX воспроизводит звук отмены. Подтверждение пути происходит в ApproveThePath, отключая ввод, а выполнение — в StartMoving, вызываемом для игрока в порядке очереди (turnPriority). Метод

CorrectingPath исключает клетки, занятые героями или боссом, проверяя через MapClass.GetMapObjectList.

Корутина MovePlayer последовательно перемещает персонажа по точкам MovementList, обновляя позицию через transform.position и синхронизируя координаты (ChangePlayerCoordinatesRpc) и состояние поля (ChangeMapStatesRpc). Задержка между шагами (timeBetweenMovement) обеспечивает плавность. После завершения MovementList очищается, и вызывается SendPlayerEndMoving. Сетевая синхронизация осуществляется через RPC (SendPlayerStartMoveRpc, SendPlayerEndMoveRpc), гарантируя согласованность позиций и событий между клиентами.

Эта система обеспечивает интуитивное управление перемещением, интегрированное с интерфейсом и сетевой логикой, позволяя игрокам стратегически планировать действия с учетом энергии и препятствий.

3.3.2 Управление игровым полем

Управление игровым полем в проекте Few Seconds – Many Deaths реализовано через класс MapClass, который обеспечивает структуру и логику клеточного поля, используемого для перемещения персонажей, размещения объектов и проверки их состояния. Поле является основой тактического геймплея, позволяя игрокам и боссу взаимодействовать с окружающей средой. Реализация включает инициализацию клеток, управление их состоянием, проверку проходимости и интеграцию с алгоритмом поиска пути, что гарантирует корректное функционирование механик перемещения и навыков.

Класс MapClass инициализируется в методе Awake, где создается двумерный массив TilesInfoArray размером $Max_A \times Max_B$, содержащий информацию о каждой клетке (доступность, объекты, эффекты). Свойство GameplayTilemap предоставляет доступ к тайловой карте Unity, а tileZero определяет смещение координат для корректного преобразования между мировыми и локальными координатами. Метод FindTerrain, вызываемый в Start,

сканирует поле, определяя проходимые клетки: если клетка в `GameplayTilemap` отсутствует, она помечается как непроходимая (`NoPlayableTile`), а проходимые клетки добавляются в список `AllTiles`. Событие `SendMapClassInitialized` уведомляет другие системы об успешной инициализации.

Состояние клеток управляется через методы, реализующие интерфейс `IMap`. Метод `SetHero` добавляет объект `Hero` с указанным `playerID` в список `MapObjectList` клетки, а `SetBoss` и `SetCorpse` выполняют аналогичные действия для босса и трупов. Методы удаления (`RemoveHero`, `RemoveBoss`, `RemoveCorpse`) очищают соответствующие объекты из списка. Для временной блокировки клеток используется `SetTempBloked`, добавляющий объект `TempBloked`, а `RemoveTempBloked` снимает блокировку. Метод `UpdateWalkability` обновляет проходимость клетки в алгоритме поиска пути (`GridPathfinding`), обеспечивая актуальность маршрутов.

Проверка состояния клеток осуществляется через `IsPlayable`, который возвращает `true`, если клетка не содержит `NoPlayableTile` и находится в пределах поля (`CheckBoundaries`). Метод `GetMapObjectList` возвращает список объектов в указанной клетке, а `MapObjectCheck` проверяет объекты в заданной области, что используется для определения целей навыков. При смерти игрока событие `PlayerDied` вызывает `RemoveHero` и `SetCorpse`, обновляя поле для отображения трупа.

Интеграция с другими системами, такими как `PlayerMovementController` и `PlayerSkillManager`, осуществляется через методы доступа к данным поля. Например, `IsPlayable` используется для проверки возможности перемещения, а `GetMapObjectList` — для предотвращения наложения персонажей. Сетевая синхронизация изменений состояния поля обеспечивается через `RPC`, вызываемые в `PlayerMovementController` (`ChangeMapStatesRpc`), что гарантирует согласованность данных между клиентами. Эта реализация обеспечивает гибкое и надежное управление игровым полем, поддерживая динамичный и тактический геймплей.

3.3.3. Система навыков персонажей

Система навыков персонажей в игре Few Seconds – Many Deaths, реализованная в классе `PlayerSkillManager`, обеспечивает игрокам возможность использовать тактические способности в стадии выбора действий (`PlayerTurnStage`). Навыки, настроенные через `SkillScript` на основе `ScriptableObject`, предоставляют гибкость в определении эффектов, зон действия и затрат энергии. Система поддерживает кооперативный геймплей благодаря сетевой синхронизации через `Netcode for GameObjects`, гарантируя согласованность действий между клиентами.

Выбор навыка осуществляется через метод `GetSkill`, активируемый нажатием клавиш (`SelectSkillByButton`) или интерфейсом. Перед использованием проверяются доступность энергии игрока (хранится в `CombatPlayerDataInStage`) и перезарядка навыка, управляемая классом `SkillCooldownManager`. Если условия соблюдены, навык добавляется в список выбранных (`SkillList`), а энергия вычитается через `ChangePlayerEnergyRpc`. Игрок указывает клетки-цели мышью, которые сохраняются в `TargetTileList` методом `AddSkillToList`. Метод `CanselAction` позволяет отменить выбор, возвращая энергию и сбрасывая перезарядку через `SendChangeSkillCooldownRpc`.

Выполнение навыков происходит в стадии результатов (`StartSkillSystem`), где метод `CastAction` последовательно активирует каждый навык из `SkillList`. Сетевая синхронизация осуществляется через `CastActionRpc`, который вызывает метод `Cast` объекта `SkillScript`, передавая позиции игрока, цели и идентификатор игрока. Это обеспечивает одновременное отображение эффектов (например, урона или лечения) для всех игроков. Перезарядка навыков уменьшается на стадии предсказания методом `SkillCooldownDecrease`, обновляя состояние через событие `SendUpdateCooldown`.

Листинг 3.4 — метод сетевого вызова выполнения навыка

```
[Rpc(SendTo.ClientsAndHost)]
```

```

private void CastActionRpc(int casterPlayerId, int skillNumber, Vector2
CasterPosition, Vector2 ActualCasterPosition, Vector2[] TargetPoints, int skillIndex)
{
    this.casterPlayerId = casterPlayerId;
    int variation = PlayerInfoData.instance.SkillChoiceList[casterPlayerId].
variationList[skillNumber];
    SkillScript skillScript = PlayerInfoData.instance.HeroDataList[casterPlayerId].
SkillList[skillNumber].SkillVariationsList[variation].SkillScript;
    CurrentSkill = skillScript;
    skillScript.Cast(CasterPosition, ActualCasterPosition, TargetPoints, casterPlayerId,
skillIndex);
}

```

Система навыков интегрирована с визуализацией (SkillVisualizer), отображающей доступные и затронутые зоны действия, что помогает игрокам принимать тактические решения. Гибкость SkillScript позволяет легко добавлять новые навыки, а сетевая синхронизация обеспечивает согласованность в кооперативной игре, усиливая взаимодействие между игроками.

3.3.4 Визуализация игрового поля и действий

Визуализация игрового поля и действий в Few Seconds – Many Deaths реализована через класс GridVisualizer, объединяющий компоненты TileManager, PathVisualizer, SkillVisualizer и BossTileManager. Эти компоненты обеспечивают динамическое отображение клеток, путей перемещения, зон действия навыков и позиции босса, помогая игрокам принимать тактические решения в кооперативной игре. Для оптимизации производительности используются пулы объектов, минимизирующие создание и удаление элементов интерфейса. События, такие как PathChanged и PlayerSkillChoosed, обеспечивают реактивное обновление визуальных элементов в реальном времени.

TileManager отвечает за отображение клеток, занятых героями, и селектора текущей клетки. Метод CreatePlayersTiles создаёт визуальные элементы для героев с использованием цветовой палитры игрока (PlayerInfoData.ColorList), а методы ShowPlayersTiles и HidePlayersTiles управляют их видимостью в зависимости от стадии игры. Селектор клетки, обновляемый в UpdateTileSelector, меняет вид в зависимости от содержимого клетки: союзник (Hero), враг (Boss), препятствие (TempBlocked) или стандартная клетка. Это позволяет игрокам быстро оценивать состояние поля.

PathVisualizer отображает маршруты перемещения персонажей. Метод UpdatePath строит временный путь от текущей позиции игрока (LastPosition) до выбранной клетки, учитывая доступную энергию. Подтверждённые пути визуализируются через CreateActualPath, используя пулы объектов (playerPathTilePool, playerPathEndTilePool) для плиток пути и конечной точки. Метод SendPathTiles синхронизирует отображение путей между клиентами через SendPlayerPathTileRpc, применяя цвет игрока для различия маршрутов в кооперативной игре.

SkillVisualizer отвечает за визуализацию зон действия навыков. Метод UpdateSkillAvaliableArea отображает доступные для выбора клетки, рассчитанные в SkillScript.AvailableTiles, а UpdateSkillAffectedArea показывает область поражения навыка на основе текущей позиции курсора. Подтверждённые зоны отображаются через UpdateAprovedAffectedAreas с использованием пула skillAprovedAffectedTilePool. Пулы объектов (skillAvaliableTilePool, skillAffectedTilePool) оптимизируют производительность, особенно при частом обновлении зон. Очистка зон происходит через методы ClearAvaliableArea и ClearAprovedAffectedAreas при смене действий или стадии.

BossTileManager управляет отображением клетки босса. Метод CreateBossTile создаёт визуальный элемент на стартовой позиции босса (BossManager.CurrentCoordinates), а методы ShowBossTile и HideBossTile синхронизируют его видимость с перемещениями босса. Это обеспечивает

чёткое представление о положении противника на поле, что критично для планирования действий в кооперативной игре.

Визуализация тесно связана с геймплеем, предоставляя игрокам обратную связь о возможных действиях. События `GlobalEventSystem` (например, `PathChanged`, `PlayerSkillApproved`) обеспечивают динамическое обновление интерфейса, а сетевая синхронизация через `Netcode for GameObjects` гарантирует согласованность отображения для всех игроков. Использование пулов объектов и оптимизированных методов визуализации позволяет поддерживать высокую производительность даже при интенсивном взаимодействии с игровым полем.

3.3.5. Архитектура боевой системы

Действия босса в игре *Few Seconds – Many Deaths*, реализованные через класс `BossManager`, обеспечивают динамичное противодействие игрокам, усиливая тактический аспект кооперативного геймплея. Босс выполняет комбинации атак, выбираемые случайным образом на стадии предсказания, с последующей синхронизацией через `Netcode for GameObjects`. Визуализация действий включает отображение призрачного спрайта босса и поворот к ближайшему игроку, что помогает игрокам прогнозировать угрозы и планировать защиту.

Выбор комбинации атак осуществляется методом `ChoiceCombo`, который случайным образом определяет комбо из списка (`ComboAttackList`) для текущей фазы боя (`CurrentAct`). Метод `GetTargetPointsForActions` задаёт координаты целей для каждой атаки в комбо, синхронизируя их через `GetTargetPointsForActionsRpc`. Выполнение атак происходит в методе `CastCombo`, вызывающем `CastActionRpc` для каждой атаки, что обеспечивает применение эффектов (например, урона или блокировки клеток) с учётом целей (`TargetPointsForActions`). Сетевая синхронизация гарантирует, что все игроки видят действия босса в реальном времени.

Визуализация реализована через призрачный спрайт (GhostBossGameObject), отображаемый на стадии предсказания методом EnableGhostBoss. Это позволяет игрокам видеть предполагаемые действия босса, такие как зоны атаки, и корректировать свои перемещения или навыки. Метод RotateBossToNearestPlayerRpc поворачивает спрайт босса (или его призрака) к ближайшему игроку, усиливая визуальную обратную связь. Позиция босса обновляется через ResetCurrentCoordinatesRpc, синхронизируя её с CurrentCoordinates.

Переход между фазами боя происходит в методе ChangeAct, активируемом при достижении нуля здоровья (CurrentHPInCurrentAct). Новая фаза обновляет характеристики босса (например, максимальное здоровье) через ChangeArcRpc, а при исчерпании всех фаз вызывается событие BossDied. Метод StopCombo скрывает призрачный спрайт на стадии игроков, завершая цикл действий босса.

Механика действий босса влияет на тактику игроков, вынуждая их избегать зон атаки и координировать действия для минимизации урона. Сетевая синхронизация и визуализация обеспечивают прозрачность и согласованность в кооперативной игре, делая босса ключевым элементом стратегического взаимодействия.

3.3 Перемещение и навыки персонажей

Навыки игроков и действия босса используют гибкую архитектуру на основе ScriptableObject, позволяющую настраивать эффекты, визуализацию и звуковое сопровождение. Классы SkillScript и BossActionScript предоставляют базовую функциональность, а их наследники реализуют конкретные механики, такие как нанесение урона, наложение щитов или перемещение. События анимаций управляются через AnimationEventManager, синхронизируя выполнение действий с визуальными эффектами. Сетевая синхронизация через

Netcode for GameObjects обеспечивает согласованность в многопользовательской среде.

3.4.1 Базовая структура навыков игроков

Класс SkillScript является базовым для всех навыков игроков в игре Few Seconds – Many Deaths, обеспечивая гибкую настройку тактических действий через архитектуру ScriptableObject. Он определяет ключевые параметры: стоимость энергии (EnergyCost), время перезарядки (SkillCooldown), количество целей (TargetCount) и флаг возможности перемещения (IsMovable). Эти параметры позволяют настраивать разнообразные эффекты, такие как атака, защита или перемещение, поддерживая стратегическое взаимодействие в кооперативном геймплее.

Основной метод Cast управляет процессом применения навыка, последовательно вызывая CastStart для инициализации параметров (позиция игрока, цели), CastFX для воспроизведения визуальных и звуковых эффектов (CastVFXPrefab, AreaVFXPrefab, castSFX) и CastEnd для завершения действия с вызовом события SendPlayerSkillEnd. Метод CastStart учитывает возможность перемещения, корректируя координаты целей через ChangeSelectedCellCoordinate, если игрок изменил позицию. Методы Area и AvailableTiles определяют зону действия навыка и доступные для выбора клетки, обеспечивая точное нацеливание.

Визуализация навыков оптимизирована с использованием пулов объектов из SkillTilePoolManager (например, attackTilePool, shieldTilePool), что минимизирует создание и удаление объектов. Метод SpawnSkillObjects размещает тайлы эффектов (TilePrefab) в зоне действия, а HideSkillObjects возвращает их в пул для повторного использования. Поддержка локализации реализована через GetLocalizeVariablesList, позволяя адаптировать текстовые описания навыков.

Взаимодействие с игровыми объектами осуществляется через методы `GetAffectedCombatObjectList` и `GetHeroCombatObject`, которые возвращают списки затронутых сущностей (Hero, Boss) для применения эффектов, таких как урон или щиты. Такая структура позволяет легко расширять систему новыми навыками, сохраняя производительность и тактическую глубину.

3.4.2 Примеры реализации навыков игроков

Навыки игроков в игре *Few Seconds – Many Deaths* реализуются через наследование от базового класса `SkillScript`, что позволяет создавать разнообразные механики с гибкой настройкой эффектов, визуализации и звукового сопровождения. Рассмотрим два примера: `HorizontalSign` и `QuickShot`, демонстрирующие различные подходы к реализации тактических действий, их визуализации и взаимодействия с игровой средой.

Навык `HorizontalSign` создаёт горизонтальную линию действия, накладывая щит на союзников в зоне. Метод `Area` формирует область действия (`CoordinateLine`), охватывающую все клетки в строке от левого края карты до правого (`MapClass.Max_A`). Навык активируется через метод `Cast`, который вызывает `CastStart` для инициализации координат, `StartAnimation` для воспроизведения анимации персонажа ("Cast2") и `CastFX` для отображения визуальных эффектов (`CastVFXPrefab`, `AreaVFXPrefab`) и звуков (`castSFX`). Метод `ApplyShield` накладывает щит (`shieldValue`) на объекты в зоне действия, определённые через `GetAffectedCombatObjectList`. Визуализация зоны осуществляется с использованием пула объектов (`shieldTilePool`) в методе `SpawnSkillObjects`, а метод `HideSkillObjects` возвращает тайлы в пул, оптимизируя производительность. Навык не зависит от боеприпасов или дополнительных условий, что делает его надёжным защитным инструментом.

Навык `QuickShot` реализует быструю атаку по одной клетке с учётом боеприпасов и бонусного урона. Метод `Area` возвращает одиночную цель (`selectedCellCoordinate`), а `AvailableTiles` предоставляет все доступные клетки

карты. В методе Cast вызываются CastStart, StartAnimation (анимация "Cast1_Part1" или "Cast1_Part2" в зависимости от порядка навыков) и CastFX для эффектов и звука (castSFX). Метод CastQuickShot проверяет наличие боеприпасов через ShotsManager и, при их наличии, запускает корутину ShotMovement, анимирующую траекторию снаряда (QuickShotPrefab) с заданной скоростью (shotSpeed). Урон (Damage) увеличивается на 50% при наличии союзников рядом (IsPlayersNear), а метод ApplyDamage наносит урон и применяет эффект кровотечения (bleedEffect) через CombatMethods. Визуализация включает эффект попадания (HitVFXPrefab) и тайлы атаки (attackTilePool). При отсутствии боеприпасов отображается сообщение (noAmmoLocalizedString), а тайлы очищаются (HideSkillObjects).

Оба навыка используют пулы объектов из SkillTilePoolManager для оптимизации, минимизируя создание и удаление визуальных элементов. Звуковые эффекты (castSFX, hitSFX) и анимации усиливают обратную связь, помогая игрокам отслеживать действия. HorizontalSign ориентирован на командную защиту, тогда как QuickShot подходит для точечных атак с тактическими решениями, зависящими от боеприпасов и позиционирования.

3.4.3 Базовая структура действий босса

Класс BossActionScript служит основой для реализации действий босса в игре Few Seconds – Many Deaths, обеспечивая гибкую настройку атак и эффектов через архитектуру ScriptableObject. Он определяет базовые компоненты для визуализации и звукового сопровождения, а также предоставляет методы для выполнения действий и взаимодействия с игровыми объектами. Действия босса синхронизируются через Netcode for GameObjects, гарантируя согласованность в кооперативной игре, а оптимизация достигается за счёт использования пулов объектов.

Класс включает параметры для визуальных эффектов (affectedTile для отображения зон поражения, hitVFX для эффектов попадания) и звуков (castSFX

для звука активации, hitSFX для звука попадания). Метод Cast инициирует действие, вызывая CastStart для настройки параметров (цели в TargetPoints, текущая фаза боя в act), CastAreaForSkill для отображения зоны поражения и CastEnd для завершения, отправляя событие BossActionEnd. Метод GetCastPoint определяет координаты целей, зависящие от фазы боя, что позволяет адаптировать действия босса к прогрессу игры.

Визуализация зон поражения адаптируется к стадии игры: на PlayerTurnStage тайлы отображаются с пониженной прозрачностью (alphaForGhost), сигнализируя игрокам о предстоящей атаке, а на BossTurnStage — с полной. Метод CastAreaForSkill использует пул объектов (SkillTilePoolManager.attackTilePool) для оптимизации, минимизируя создание новых тайлов. Метод DestroyAffectedTilesPrefabs очищает неактивные тайлы, снижая нагрузку на производительность.

Для взаимодействия с игровыми объектами метод GetAffectedCombatObjectList возвращает список затронутых сущностей (Hero, Boss) в указанной зоне, а DamageEveryOneInTiles наносит урон всем объектам в клетках, сопровождая это звуковыми и визуальными эффектами. Метод GetPoint обеспечивает точное позиционирование эффектов (например, hitVFX) относительно центра объекта. Метод CastPartOfAction поддерживает многоэтапные действия, позволяя разбивать сложные атаки на последовательные шаги, синхронизированные с анимациями через StartAnimation.

BossActionScript обеспечивает модульность и расширяемость, позволяя создавать разнообразные действия босса (например, атаки по площади или рывки) с минимальными изменениями кода.

3.4.4 Примеры реализации действий босса

Действия босса в игре Few Seconds – Many Deaths реализуются через наследников класса BossActionScript, обеспечивая разнообразные тактические угрозы для игроков. Рассматриваются два примера: BerserkCircularSlice_Berserk

и Dash_Berserk, демонстрирующие нанесение урона, перемещение и взаимодействие с игровым полем. Эти действия используют анимации, звуковые эффекты и визуализацию зон поражения, синхронизируемые через Netcode for GameObjects для согласованности в кооперативной игре.

Класс BerserkCircularSlice_Berserk реализует круговую атаку босса, создающую зону поражения с радиусом, зависящим от фазы боя (act). Метод GetCastPoint возвращает центр атаки (координаты босса), а метод Cast иницирует действие, вызывая CastCircularSlice через CastPartOfAction с индексом 1. Зона поражения формируется методом SquareAOE, отображаемым через CastAreaForSkill с использованием пула attackTilePool. Урон наносится игрокам в зоне (DamagePlayers), а босс восстанавливает здоровье (ApplyHeal) пропорционально нанесённому урону (healModifier). Визуализация включает анимацию (StartAnimation, триггер "Circle") и звуковые эффекты (castSFX, healSFX), с прозрачностью тайлов, адаптированной для стадии предсказания (alphaForGhost).

Листинг 3.5 — метод исполнения круговой атаки босса

```
private void CastCircularSlice()
{
    int _radius = radius;
    if (act > 1) _radius = radiusUpgrade + radius;
    List<Vector2> CircularList = GridAreaMethods.SquareAOE
(bossManager.CurrentCoordinates, bossManager.CurrentCoordinates, _radius, true);
    CastAreaForSkill(CircularList);
    DamagePlayers(CircularList);
}
```

Класс Dash_Berserk реализует рывок босса через последовательность случайных клеток, заданных методом GetCastPoint. В фазе боя выше первой

выбираются случайные клетки (randomCellCount), а последняя цель — точка рядом с игроком (GetRandomPointNearRandomPlayer). Метод Cast запускает корутину Dash, которая строит пути (GridPathfinding.FindPath) и корректирует их (CorrectingPath), избегая коллизий с героями или боссом. Движение анимируется через MoveAlongPath с убывающей скоростью (StartDashSpeed, SpeedDecreaseMultiplayer), нанося урон (DamageEveryOneInTiles) на каждой конечной клетке в стадии BossTurnStage. Визуализация включает анимацию рывка (StartDash), звуковые эффекты (castSFX, hitSFX) и поворот спрайта (RotateBossToMovementDirection). Позиция босса обновляется через MapClass.SetBoss, синхронизируя координаты (CurrentCoordinates).

Оба действия усиливают тактический аспект, вынуждая игроков предсказывать зоны поражения и координировать перемещения.

3.4.5 Управление событиями анимаций

Управление событиями анимаций в проекте Few Seconds – Many Deaths реализовано через класс AnimationEventManager, обеспечивающий синхронизацию визуальных эффектов с выполнением навыков игроков и действий босса. Этот класс играет ключевую роль в согласовании анимаций с игровыми событиями, что усиливает визуальную обратную связь и поддерживает динамику кооперативного геймплея.

Метод SetSkillIndexAndStartActions управляет анимациями навыков игроков, вызывая метод CastPartOfSkill текущего навыка (CurrentSkill) из PlayerSkillManager. Этот метод позволяет запускать определённые этапы анимации, например, завершение действия в навыке QuickShot, где анимация делится на части (Cast1_Part1, Cast1_Part2, Cast1_Part3) для точного соответствия игровым эффектам, таким как выстрел или нанесение урона. Проверка на наличие текущего навыка с помощью Debug.LogError предотвращает ошибки при отсутствии активного навыка, обеспечивая надёжность системы.

Метод `SetActionIndexAndStartActions` отвечает за анимации действий босса, вызывая `CastPartOfAction` для текущего действия из списка `BossActionList` в `BossManager`. Например, в `BerserkCircularSlice_Berserk` метод запускает этап анимации круговой атаки (`Circle`), а в `Dash_Berserk` — этапы рывка (`BerserkDash_Loop`). Метод учитывает индекс текущего действия (`CurrentAction`) и проверяет корректность комбо, логируя ошибки (`Debug.LogError`) при несоответствии, что упрощает отладку.

Точное соответствие анимаций игровым событиям, таким как нанесение урона или перемещение, повышает восприятие тактических решений, делая игровой процесс более интуитивным и зрелищным.

3.4.6 Сетевая синхронизация и оптимизация

Сетевая синхронизация навыков игроков и действий босса в игре *Few Seconds – Many Deaths* реализована с использованием `Netcode for GameObjects`, обеспечивая согласованность игровых событий в кооперативной многопользовательской среде. Оптимизация производительности достигается за счёт применения пулов объектов и эффективного управления ресурсами, минимизируя нагрузку на клиентские устройства.

Для навыков игроков сетевая синхронизация осуществляется через метод `CastActionRpc` в классе `PlayerSkillManager`. Этот метод передаёт параметры навыка, включая идентификатор игрока (`casterPlayerId`), номер навыка (`skillNumber`), позицию кастера (`CasterPosition`), актуальную позицию (`ActualCasterPosition`), координаты целей (`TargetPoints`) и индекс навыка (`skillIndex`). Вызов `CastActionRpc` активирует метод `Cast` соответствующего объекта `SkillScript`, применяя эффекты (например, урон в `QuickShot` или щит в `HorizontalSign`) на всех клиентах. Это гарантирует, что визуальные и игровые эффекты навыков, такие как анимации и изменения состояния объектов, отображаются синхронно для всех игроков.

Действия босса синхронизируются через методы `CastActionRpc` и `GetTargetPointsForActionsRpc` в классе `BossManager`. Метод `GetTargetPointsForActionsRpc` передаёт координаты целей (`TargetPoints`) для каждой атаки в комбо, обеспечивая их отображение на стадии предсказания (`PlayerTurnStage`) через призренный спрайт (`GhostBossGameObject`). Метод `CastActionRpc` вызывает `Cast` объекта `BossActionScript`, применяя эффекты (например, урон в `BerserkCircularSlice_Berserk` или перемещение в `Dash_Berserk`) с учётом текущей фазы боя (`act`) и целей. События `SendBossActionEnd` и `SendTargetPointsForActionsChoosed` завершают синхронизацию, уведомляя клиентов о выполнении действий.

Оптимизация визуализации достигается за счёт использования пулов объектов, реализованных в `SkillTilePoolManager`. Для навыков игроков пулы (`attackTilePool`, `shieldTilePool`, `healTilePool`, `teleportTilePool`) предоставляют тайлы для отображения зон действия и эффектов, минимизируя создание и удаление объектов. Метод `HideSkillObjects` возвращает неактивные тайлы в пул, как видно в `HorizontalSign` и `QuickShot`. Для действий босса метод `DestroyAffectedTilesPrefabs` очищает тайлы атаки (`affectedTile`) после завершения действия, как в `BerserkCircularSlice_Berserk`. Это снижает нагрузку на память и процессор, особенно при частом применении навыков и атак.

Дополнительная оптимизация реализована через адаптивную визуализацию зон действия. В `BossActionScript` метод `CastAreaForSkill` изменяет прозрачность тайлов (`alphaForGhost`) на стадии предсказания, снижая визуальную нагрузку. В `SkillScript` методы `SpawnSkillObjects` и `CastVFX` проверяют наличие тайлов на карте (`GameplayTilemap.HasTile`), предотвращая создание объектов на непроходимых клетках. Сетевая синхронизация минимизирует избыточные вызовы RPC, передавая только необходимые данные, такие как координаты и индексы.

Сочетание сетевой синхронизации и оптимизации обеспечивает плавный кооперативный геймплей, позволяя игрокам координировать действия против босса. Эффективное управление ресурсами поддерживает стабильную

производительность даже при интенсивных игровых событиях, таких как множественные атаки или массовые эффекты навыков.

Заключение

В ходе выпускной квалификационной работы была разработана компьютерная игра Few Seconds – Many Deaths, представляющая собой тактическую кооперативную игру с клеточным полем, системой навыков и динамичными действиями босса. Программный проект обеспечивает прочную основу для многопользовательской 2D-игры, включающей механики перемещения, управления навыками, визуализации игрового поля и сетевой синхронизации. Реализованные скрипты, основанные на модульной архитектуре с использованием ScriptableObject и Netcode for GameObjects, демонстрируют гибкость и масштабируемость, что упрощает добавление новых функций, таких как дополнительные навыки или действия босса.

В процессе создания программного продукта были успешно выполнены этапы анализа, проектирования, разработки и тестирования, что позволило достичь полного соответствия поставленным требованиям. В ходе работы выполнения были достигнуты следующие поставленные задачи:

- Проведено исследование предметной области тактических кооперативных игр;
- Выполнен анализ аналогичных игровых продуктов;
- Спроектирована архитектура программной системы, включая модули управления полем, навыками и действиями босса;
- Разработана кооперативная 2D-игра с системой перемещения, навыков, визуализации и сетевой синхронизацией.

Список литературы

1. Макконнелл С. Совершенный код. Практическое руководство по разработке программного обеспечения. — СПб.: Символ-Плюс, 2020
2. Шрайбер И. Unity. Разработка 2D-игр. — М.: ДМК Пресс, 2019.
3. Шелл Д. Геймдизайн. Теория и практика разработки компьютерных игр. — М.: Вильямс, 2021.
4. Шелл Д. Геймдизайн. Как создать игру, в которую будут играть все, 2008. — с. 257–478.
5. Шрейер ДЖ. Кровь, пот и пиксели. Обратная сторона индустрии видеоигр 2-е издание, 2017.
6. C Sharp Documentation [Электронный ресурс] URL: <https://learn.microsoft.com/en-us/dotnet/csharp/> (дата обращения: 28.03.2025).
7. MSDN. Объектно-ориентированное программирование на C# [Электронный ресурс] URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/> (дата обращения: 03.03.2025).
8. Newzoo. Global Games Market Report [Электронный ресурс] URL: <https://newzoo.com/> (дата обращения: 15.04.2025).
9. Unity Technologies. Unity Manual. [Электронный ресурс] URL: <https://docs.unity3d.com/Manual/index.html> (дата обращения: 28.03.2025).
10. Unity Technologies. Unity Scripting API. [Электронный ресурс] URL: <https://docs.unity3d.com/ScriptReference/> (дата обращения: 28.03.2025).
11. Unity Netcode for GameObjects Documentation. [Электронный ресурс]. — URL: <https://docs.unity3d.com/2023.2/Documentation/Manual/com.unity.netcode.gameobjects.html> (Дата обращения: 03.04.2025)
12. Unity Transport Documentation. [Электронный ресурс]. — URL: <https://docs.unity3d.com/2023.1/Documentation/Manual/com.unity.transport.html> (Дата обращения: 03.04.2025)

Приложение А. Техническое задание

1. Концепция игры

1.1 Описание игры

Игра Few Seconds – Many Deaths — кооперативная тактическая 2D-игра в жанре Rogue-like с пошаговой боевой системой, разработанная на Unity 6 для ПК. Игроки (до четырёх человек) управляют уникальными персонажами с ролями (атака, защита, лечение, стратегия), совместно сражаясь с боссом на клеточном поле. Основной акцент — на командной синергии, тактическом планировании и высокой реиграбельности за счет случайной очередности ходов и динамичных действий босса.

1.2 Характеристики игры

Платформа: PC, мобильные устройства

Жанры: Rogue-like, Cooperative, Turn-based

Сеттинг: Fantasy

Графика: 2D, Pixel Art, Top Down

ПО: Unity Engine, Visual Studio

Целевая аудитория: от 14 до 30 лет

Продолжительность: 1 час

1.3 Системные требования

Таблица 1 демонстрирует системные минимальные требования к разрабатываемой игре.

Таблица 1 - Системные требования

Система	Минимальное требование
---------	------------------------

Операционная система	Windows 10 / 11
Процессор	Двухъядерный, от 2.5 GHz
Оперативная память	От 8 GB
Видеокарта	С поддержкой DirectX 11
Разрешение	1280×720 и выше

1.4 Управление

Кнопки на клавиатуре отвечают за действия, приведенные в таблице 2.

Таблица 2 - Управление с клавиатуры

Клавиша	Действие
Esc	Открыть меню
Space	Завершение хода / Пропуск катсцены
Q	Скилл Q
W	Скилл W
E	Скилл E
R	Скилл R

Чтобы выбрать клетку для передвижения / места назначения скилла, нужно нажать на нее ЛКМ.

2. Игровой контент

2.1 Игровая сцена

Игровой мир представляет собой клеточное поле (размером $Max_A \times Max_B$), процедурно генерируемое с препятствиями и проходимыми зонами. Поле визуализируется через Unity Tilemap с динамическим обновлением состояния клеток (занятость героями, боссом, временная блокировка). Атмосфера игры поддерживается стилизованной 2D-графикой и звуковыми эффектами, создающими напряженную тактическую обстановку.

2.2 Игровой цикл

Игровой цикл состоит из последовательных стадий боя:

- **Предсказание:** Босс демонстрирует будущие атаки через призрачный спрайт, игроки восстанавливают энергию, сбрасываются щиты.
- **Выбор действий:** Игроки одновременно выбирают перемещения или навыки в ограниченное время, подтверждают действия через интерфейс.
- **Выполнение действий игроков:** Действия выполняются в случайном порядке, синхронизируются через RPC.
- **Действия босса:** Босс выполняет выбранные атаки, нанося урон или применяя эффекты, завершая цикл. Цикл повторяется до победы над боссом или гибели всех игроков. События завершения боя отображаются через меню победы/поражения.

2.3 Игровая механика

- **Перемещение:** Игроки перемещаются по клеточному полю, расходуя энергию (2 единицы за горизонтальное/вертикальное движение, 3 — за диагональное). Алгоритм GridPathfinding строит маршруты, исключая занятые клетки.

- **Система навыков:** Навыки игроков включают атаку, защиту, лечение и другие эффекты. Затраты энергии, перезарядка и зоны действия настраиваются через ScriptableObject.
- **Действия босса:** Босс выполняет случайные комбинации атак, такие как круговая атака или рывок, с изменением поведения по фазам боя.
- **Случайная очередность ходов:** TurnOrderManager случайно определяет порядок действий игроков в каждом цикле, синхронизируя его через Rpc.
- **Сетевая синхронизация:** Используется Netcode for GameObjects для синхронизации перемещений, стадий боя, навыков и атак.
- **Визуализация:** Зоны действия и эффекты отображаются через пулы объектов, оптимизируя производительность. Призрачный спрайт босса визуализирует атаки на стадии предсказания.

Приложение Б. Презентация

Федеральное государственное автономное
образовательное учреждение высшего образования
«Московский политехнический университет»

Разработка сетевой составляющей и пошагового боевого режима в Rogue-like игре

Автор: Киселева Екатерина Романовна
Студент группы 211-727
Научный руководитель: Арсентьев Д.А.



Введение

Актуальность проекта:

1. Потребность рынка в играх жанра Rogue-like
2. Меньшая конкуренция в жанре, если рассматривать игры с кооперативной составляющей
3. Спрос на графику Pixel Art



Введение

Цель работы:

Разработка сетевой составляющей и пошагового боевого режима в Rogue-like игре



Общее описание игры

Платформа: PC

Основные жанры:

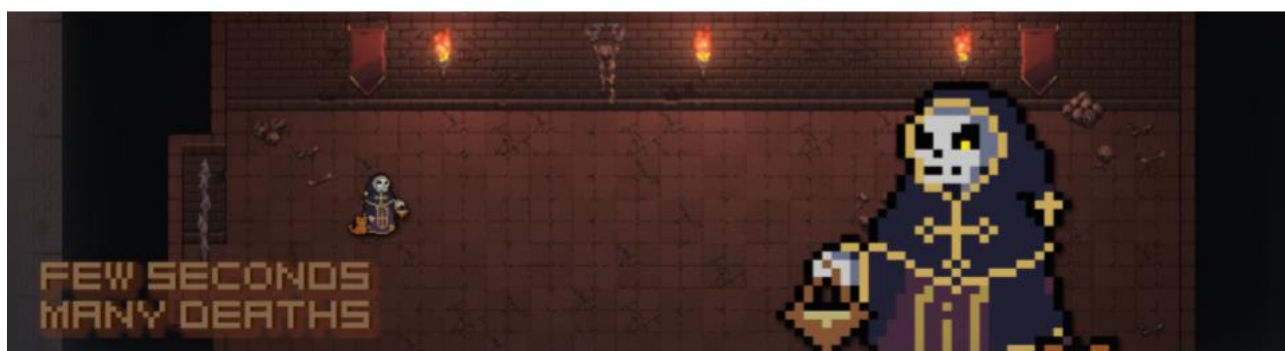
Coop/Rogue-like/Turn-Based

Сеттинг: Fantasy ("Фэнтези")

Графика: Top-Down / Pixel Art / 2D

Движок: Unity

Мультиплеер: Netcode for
GameObjects + Unity Transport



Основные элементы игры

- Система пошагового боя
- Система передвижения
- Скиллы героев
- Атаки босса
- Статус эффекты



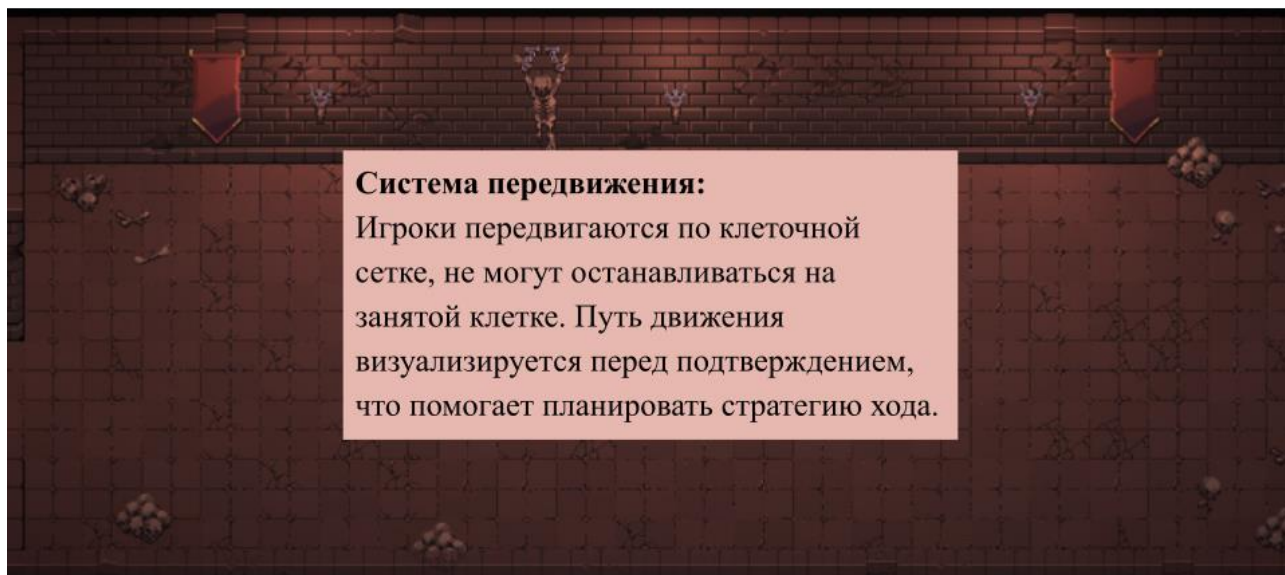
Основные элементы игры

Пошаговая система:

Игроки по очереди выбирают действия, учитывая энергию и способности, с учетом синхронизации порядка ходов через Turn Order Manager.



Основные элементы игры



Основные элементы игры

Скиллы героев:

Каждый персонаж имеет уникальные способности, соответствующие его роли: атакующий наносит урон, защитник накладывает щиты, целитель восстанавливает здоровье, а стратег управляет передвижением союзников.



Основные элементы игры

Атаки босса:

Атаки босса предсказываются перед ходом игроков. Босс показывает свои атаки, позволяя игрокам стратегически планировать действия для избежания урона и применения контрмер.



Заключение

Выводы: Реализованы основные механики игры, включая пошаговый бой, кооперативное взаимодействие и систему очередности ходов.

Перспективы: Планируется расширение контента, улучшение графики и добавление новых уровней.

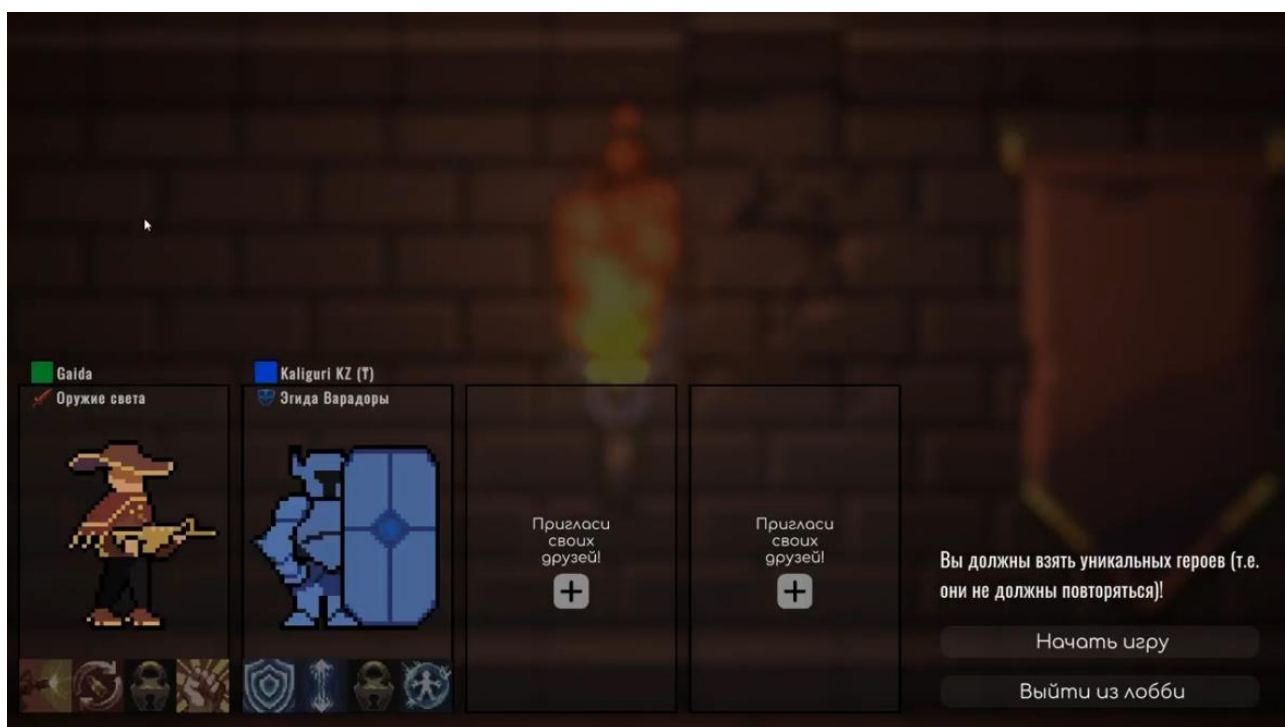
Значимость работы: Проект представляет ценность для образовательного процесса, предлагает увлекательный игровой опыт и является примером успешной инди-разработки.

Спасибо за внимание!

Автор: Киселева Екатерина Романовна

Студент группы 211-727

Научный руководитель: Арсентьев Д.А.



Приложение В. Листинг скриптов управления игровыми сессиями

```
using UnityEngine;
using Unity.Services.Core;
using Unity.Services.Authentication;
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine.UI;
using System.Threading.Tasks;
using System;
using Unity.Services.Multiplayer;
using Unity.Netcode;
using PlayEveryWare.EpicOnlineServices.Samples;

public class SessionsManager : MonoBehaviour
{
    [Header("UI")]
    [SerializeField] GameObject SessionSearchMenu;
    [SerializeField] GameObject InLobbyMenu;
    [SerializeField] GameObject MainMenu;
    [SerializeField] TMP_Text passwordText;
    [SerializeField] TMP_Text JoinCode;
    [SerializeField] GameObject warningText;
    [SerializeField] GameObject passwordPanel;
    [SerializeField] GameObject LobbyInfoPanel;
    [SerializeField] Transform sessionListContainer;
    [SerializeField] GameObject sessionItemPrefab;
    [SerializeField] GameObject passwordPromptPanel;
    [SerializeField] TMP_Text errorMessageText;
    [SerializeField] TMP_InputField joinCodeInputField;
```

```
[SerializeField] TMP_InputField passwordInput;
[SerializeField] Button confirmPasswordButton;
[SerializeField] Toggle isPrivate;
[SerializeField] Button copyJoinCodeButton;
[SerializeField] Button copyPasswordButton;
[SerializeField] Button cancelPasswordButton;
```

```
[Header("ControllerSession")]
[SerializeField] GameObject ControllersSessionPrefab;
```

```
private ISessionService sessionService;
private ISessionUI sessionUI;
```

```
private string sessionName = "Session";
private string sessionMod = "BossRush";
private int maxPlayers = 4;
private string lobbyPassword = "";
private ISessionInfo pendingSession;
private GameObject ControllersSession;
private Coroutine sessionUpdateCoroutine;
private ObjectsPool sessionPool;
```

```
private float sessionUpdateInterval = 10f;
public static SessionsManager instance = null;
```

```
private void Awake()
{
    if (instance == null) instance = this;
```

```

        sessionPool = new ObjectsPool(sessionItemPrefab, 20, new
GameObject("sessionItemParent").transform);
        sessionService = new SessionService();
        sessionUI = new SessionUI(
            SessionSearchMenu, InLobbyMenu, MainMenu, passwordText, JoinCode,
passwordPanel, LobbyInfoPanel,
            sessionListContainer, sessionPool, passwordPromptPanel, errorMessageText,
passwordInput, confirmPasswordButton,
            copyJoinCodeButton, copyPasswordButton, cancelPasswordButton
        );
    }

    private async void Start()
    {
        try
        {
            await UnityServices.InitializeAsync();
            await AuthenticationService.Instance.SignInAnonymouslyAsync();
            Debug.Log($"Sign in anonymously succeeded! PlayerID:
{AuthenticationService.Instance.PlayerId}");
        }
        catch (Exception e)
        {
            Debug.LogException(e);
        }
    }

    public async void StartSessionAsHost()
    {
        var sessionProperties = new Dictionary<string, SessionProperty>

```

```

{
    { "StringIndex1", new SessionProperty(sessionMod) }
};

var options = new SessionOptions
{
    IsPrivate = isPrivate.isOn,
    MaxPlayers = maxPlayers,
    Name = sessionName + UnityEngine.Random.Range(0, 100),
    Password = passwordInput.text == "" ? null :
FormatPassword(passwordInput.text),
    SessionProperties = sessionProperties
}.WithRelayNetwork();

lobbyPassword = passwordInput.text == "" ? null : passwordInput.text;

var session = await sessionService.CreateSessionAsync(options);
InitialiseSession(session);
}

public void SetSessionMod(string mod)
{
    sessionMod = mod;
}

private void InitialiseSession(ISession session)
{
    if (session == null)
    {
        Debug.LogError("Failed to join session: session is null");
    }
}

```

```

        return;
    }

    sessionUI.ShowErrorMessage("");
    LobbySaver.instance.currentSession = session;
    ControllersSession = Instantiate(ControllersSessionPrefab);
    sessionUI.SetUIState(UIState.InLobby);
    sessionUI.UpdateLobbyInfo(session, lobbyPassword);
    GlobalEventSystem.SendPlayerLobbyUpdate();
}

public async void LeaveLobby()
{
    await sessionService.LeaveLobby();
    NetworkManager.Singleton.Shutdown();
    PlayerInfoData.instance.RemoveNetworkManagerListeners();
    Destroy(ControllersSession);
    sessionUI.SetUIState(UIState.MainMenu);
}

public async void JoinSessionByJoinCode()
{
    if (sessionUpdateCoroutine != null)
    {
        StopCoroutine(sessionUpdateCoroutine);
        sessionUpdateCoroutine = null;
    }

    string code = joinCodeInputField.text;

```

```

if (!string.IsNullOrEmpty(code))
{
    var session = await sessionService.JoinSessionByCodeAsync(code);
    InitialiseSession(session);
}
}

```

```

public async void JoinSessionByID(string sessionId, string password)
{
    if (sessionUpdateCoroutine != null)
    {
        StopCoroutine(sessionUpdateCoroutine);
        sessionUpdateCoroutine = null;
    }

    if (string.IsNullOrEmpty(sessionId))
    {
        Debug.LogError("Session ID is null or empty!");
        return;
    }

    try
    {
        var options = new JoinSessionOptions { Password =
FormatPassword(password) };
        var session = await sessionService.JoinSessionByIdAsync(sessionId, options);

        if (session == null)
        {
            Debug.LogError("Failed to join session: session is null.");

```

```

        return;
    }

    InitialiseSession(session);
}
catch (Exception e)
{
    Debug.LogError($"Error while joining session:
{e.Message}\n{e.StackTrace}");
}
}

public async void JoinSessionWithPassword()
{
    string enteredPassword = passwordInput.text;

    if (pendingSession != null)
    {
        bool success = await TryJoinSession(pendingSession.Id, enteredPassword);
        if (success)
        {
            sessionUI.HidePasswordPrompt();
        }
        else
        {
            sessionUI.ShowErrorMessage("Incorrect password! Please try again.");
        }
    }
}
}

```

```

private async Task<bool> TryJoinSession(string sessionId, string password)
{
    try
    {
        var options = new JoinSessionOptions { Password = password };
        var session = await sessionService.JoinSessionByIdAsync(sessionId, options);

        if (session != null)
        {
            InitialiseSession(session);
            return true;
        }
        else
        {
            return false;
        }
    }
    catch (Exception e)
    {
        Debug.LogError($"Error joining session: {e.Message}");
        return false;
    }
}

private string FormatPassword(string input)
{
    if (string.IsNullOrEmpty(input))
        return null;

    if (input.Length < 8)

```

```

        input = input.PadRight(8, 'x');

        return input;
    }

    public void JoinSession(ISessionInfo session)
    {
        if (session.HasPassword)
        {
            pendingSession = session;

            sessionUI.ShowPasswordPrompt();
        }
        else JoinSessionByID(session.Id, null);
    }

    public void StartSessionSearch()
    {
        sessionUpdateCoroutine ??= StartCoroutine(UpdateSessionListCoroutine());
    }

    private IEnumerator UpdateSessionListCoroutine()
    {
        while (true)
        {
            Debug.Log("search Sessions");
            StartCoroutine(UpdateSessionList());
            yield return new WaitForSeconds(sessionUpdateInterval);
        }
    }
}

```

```

private IEnumerator UpdateSessionList()
{
    var task = sessionService.GetSessionsList();

    yield return new WaitUntil(() => task.IsCompleted);

    if (task.Exception != null)
    {
        Debug.LogError($"Get session list ended with error: {task.Exception}");
    }
    else
    {
        Debug.Log(task.Result.Sessions.Count);
        if (task.Result != null)
        {
            IList<ISessionInfo> sessions = task.Result.Sessions;
            sessionUI.UpdateSessionList(sessions);
        }
    }
}

public void StopSessionSearch()
{
    if (sessionUpdateCoroutine != null)
    {
        StopCoroutine(sessionUpdateCoroutine);
        sessionUpdateCoroutine = null;
    }
}

```

```

private bool HasDuplicates(List<HeroTypeData> list)
{
    HashSet<HeroTypeData> uniqueElements = new HashSet<HeroTypeData>();

    foreach (HeroTypeData item in list)
    {
        if (!uniqueElements.Add(item))
        {
            return true;
        }
    }
    return false;
}

public void StartGameServer()
{
    if (NetworkManager.Singleton.IsHost)
    {
        List<HeroTypeData> heroesList = new();
        for (int i = 0; i < PlayerInfoData.instance.PlayerCount; i++)
            heroesList.Add(PlayerInfoData.instance.HeroDataList[i].HeroTypeData);
        if (!HasDuplicates(heroesList))
            SceneManager.instance.LoadScene(MainMenuSceneContainer.instance.FirstSessionScene, true);
        else
        {
            warningText.SetActive(true);
        }
    }
}

```

```

    }
}

using TMPro;
using UnityEngine;
using UnityEngine.UI;
using System.Collections.Generic;
using Unity.Netcode;
using Unity.Services.Multiplayer;

public interface ISessionUI
{
    void UpdateSessionList(ICollection<ISessionInfo> sessions);
    void ShowPasswordPrompt();
    void HidePasswordPrompt();
    void ShowErrorMessage(string message);
    void UpdateLobbyInfo(ISession session, string password);
    void SetUIState(UIState state);
}

public class SessionUI : ISessionUI
{
    private Button copyJoinCodeButton;
    private Button copyPasswordButton;
    private GameObject SessionSearchMenu;
    private GameObject InLobbyMenu;
    private GameObject MainMenu;
    private TMP_Text passwordText;
    private TMP_Text JoinCode;
    private GameObject passwordPanel;

```

```

private GameObject LobbyInfoPanel;
private Transform sessionListContainer;
private ObjectsPool sessionPool;
private GameObject passwordPromptPanel;
private TMP_Text errorMessageText;
private TMP_InputField passwordInput;
private Button confirmPasswordButton;
private Button cancelPasswordButton;

```

```

public SessionUI(GameObject sessionSearchMenu, GameObject inLobbyMenu,
GameObject mainMenu, TMP_Text passwordText, TMP_Text joinCode, GameObject
passwordPanel, GameObject lobbyInfoPanel, Transform sessionListContainer,
ObjectsPool sessionPool, GameObject passwordPromptPanel, TMP_Text
errorMessageText, TMP_InputField passwordInput, Button confirmPasswordButton,
Button copyJoinCodeButton, Button copyPasswordButton, Button
cancelPasswordButton)

```

```

{
    SessionSearchMenu = sessionSearchMenu;
    InLobbyMenu = inLobbyMenu;
    MainMenu = mainMenu;
    this.passwordText = passwordText;
    JoinCode = joinCode;
    this.passwordPanel = passwordPanel;
    LobbyInfoPanel = lobbyInfoPanel;
    this.sessionListContainer = sessionListContainer;
    this.sessionPool = sessionPool;
    this.passwordPromptPanel = passwordPromptPanel;
    this.errorMessageText = errorMessageText;
    this.passwordInput = passwordInput;
    this.confirmPasswordButton = confirmPasswordButton;
}

```

```

this.copyJoinCodeButton = copyJoinCodeButton;
this.copyPasswordButton = copyPasswordButton;
this.canselPasswordButton = canselPasswordButton;

if (copyJoinCodeButton != null) copyJoinCodeButton.onClick.AddListener(() =>
CopyText(JoinCode));
if (copyPasswordButton != null) copyPasswordButton.onClick.AddListener(()
=> CopyText(passwordText));
if (canselPasswordButton != null)
canselPasswordButton.onClick.AddListener(CancelPasswordPrompt);
}

public void UpdateSessionList(ICollection<ISessionInfo> sessions)
{
    sessionPool.ReturnAllActive();

    foreach (var session in sessions)
    {
        GameObject sessionItem = sessionPool.Get();
        sessionItem.transform.SetParent(sessionListContainer, false);

        sessionItem.transform.Find("SessionName").GetComponent<TMP_Text>().text =
session.Name;

        TMP_Text isPrivateText =
sessionItem.transform.Find("IsPrivate").GetComponent<TMP_Text>();
        isPrivateText.text = session.HasPassword ? "Yes" : "No";
    }
}

```

```
sessionItem.transform.Find("PlayerCount").GetComponent<TMP_Text>().text =
$"{session.MaxPlayers - session.AvailableSlots}/{session.MaxPlayers}";
```

```
    Button joinButton = sessionItem.GetComponent<Button>();
    joinButton.onClick.RemoveAllListeners();
    joinButton.onClick.AddListener(() => OnJoinSession(session));
```

```
LayoutRebuilder.ForceRebuildLayoutImmediate(sessionListContainer.gameObject.GetComponent<RectTransform>());
    Canvas.ForceUpdateCanvases();
}
}
```

```
private void OnJoinSession(ISessionInfo session)
{
    SessionsManager.instance.JoinSession(session);
}
```

```
public void ShowPasswordPrompt()
{
    passwordPromptPanel.SetActive(true);
    passwordInput.text = "";
    errorMessageText.gameObject.SetActive(false);

    confirmPasswordButton.onClick.RemoveAllListeners();
    confirmPasswordButton.onClick.AddListener(() =>
SessionsManager.instance.JoinSessionWithPassword());
}
```

```
public void HidePasswordPrompt()
{
    passwordPromptPanel.SetActive(false);
}

public void ShowErrorMessage(string message)
{
    errorMessageText.text = message;
    errorMessageText.gameObject.SetActive(true);
}

public void UpdateLobbyInfo(ISession session, string password)
{
    if (NetworkManager.Singleton.IsHost)
    {
        LobbyInfoPanel.SetActive(true);
        JoinCode.text = session.Code;

        if (session.HasPassword)
        {
            passwordPanel.SetActive(true);
            passwordText.text = password;
        }
        else passwordPanel.SetActive(false);
    }
    else LobbyInfoPanel.SetActive(false);
}

public void CopyText(TMP_Text text)
```

```

{
    GUIUtility.systemCopyBuffer = text.text;
}

public void CancelPasswordPrompt()
{
    errorMessageText.gameObject.SetActive(false);
    passwordPromptPanel.SetActive(false);
}

public void SetUIState(UIState state)
{
    MainMenu.SetActive(state == UIState.MainMenu);
    SessionSearchMenu.SetActive(state == UIState.SessionSearch);
    InLobbyMenu.SetActive(state == UIState.InLobby);
}
}

public enum UIState
{
    MainMenu,
    SessionSearch,
    InLobby
}

using Unity.Services.Core;
using Unity.Services.Authentication;
using System.Threading.Tasks;
using System.Collections.Generic;

```

```
using Unity.Services.Multiplayer;
using UnityEngine;
using System;
```

```
public interface ISessionService
{
    Task<ISession> CreateSessionAsync(SessionOptions options);
    Task<ISession> JoinSessionByCodeAsync(string code);
    Task<ISession> JoinSessionByIdAsync(string sessionId, JoinSessionOptions
options);
    Task<QuerySessionsResults> QuerySessionsAsync(QuerySessionsOptions
options);
    Task<List<string>> GetJoinedSessionIdsAsync();
    Task ReconnectToSessionAsync(string id);
    Task LeaveLobby();
    Task<QuerySessionsResults> GetSessionsList();
}
```

```
public class SessionService : ISessionService
{
    public async Task<ISession> CreateSessionAsync(SessionOptions options)
    {
        return await MultiplayerService.Instance.CreateSessionAsync(options);
    }

    public async Task<ISession> JoinSessionByCodeAsync(string code)
    {
        return await MultiplayerService.Instance.JoinSessionByCodeAsync(code);
    }
}
```

```

public async Task<ISession> JoinSessionByIdAsync(string sessionId,
JoinSessionOptions options)
{
    return await MultiplayerService.Instance.JoinSessionByIdAsync(sessionId,
options);
}

```

```

public async Task<QuerySessionsResults>
QuerySessionsAsync(QuerySessionsOptions options)
{
    return await MultiplayerService.Instance.QuerySessionsAsync(options);
}

```

```

public async Task<List<string>> GetJoinedSessionIdsAsync()
{
    return await MultiplayerService.Instance.GetJoinedSessionIdsAsync();
}

```

```

public async Task ReconnectToSessionAsync(string id)
{
    await MultiplayerService.Instance.ReconnectToSessionAsync(id);
}

```

```

public async Task LeaveLobby()
{
    await LobbySaver.instance.currentSession.LeaveAsync();
}

```

```

public async Task<QuerySessionsResults> GetSessionsList()
{

```

```

try
{
    var options = new QuerySessionsOptions();
    QuerySessionsResults querySessionsResults = await
MultiplayerService.Instance.QuerySessionsAsync(options);
    return querySessionsResults;
}
catch (Exception e)
{
    Debug.LogError($"Catch error then trying to get sessions list: {e.Message}");
    return null;
}
}

```

Приложение Б. Листинг скриптов архитектуры боевой системы

```
using System.Collections;
using System.Collections.Generic;
using Unity.Netcode;
using Unity;
using Unity.VisualScripting;
using UnityEngine;
using TMPro;
using UnityEngine.UI;
using UnityEngine.Localization.Settings;
using System;
using UnityEngine.Tilemaps;

public class CombatStageManager : NetworkBehaviour
{
    public float PlayerTurnTime;
    [SerializeField] private TextMeshProUGUI timerText;
    [SerializeField] private TextMeshProUGUI confirmationText;
    [SerializeField] private Button confirmationButton;
    [SerializeField] private GameObject WinMenu;
    [SerializeField] private GameObject LoseMenu;

    public GameState currentStage = null;
    private int currentStageIndex = -1;
    private NetworkVariable<int> currentStageIndexNet = new();

    private List<GameState> _stages;
    public bool isCombatRunning = true;
    public static CombatStageManager instance = null;
```

```

private void Awake()
{
    if (instance == null) instance = this;
    GlobalEventSystem.PlayerDataUpdated.AddListener(StartBattle);
    GlobalEventSystem.AllPlayersDied.AddListener(LoseBattle);
    GlobalEventSystem.BossDied.AddListener(WinBattle);
}

public override void OnNetworkSpawn()
{
    base.OnNetworkSpawn();
    Initialize();
}

private void Update()
{
    if (currentStageIndexNet.Value != currentStageIndex &&
!NetworkManager.Singleton.IsServer)
    {
        NextStage(currentStageIndexNet.Value);
    }

    if (currentStage != null)
    {
        currentStage.UpdateStage();
    }
}

private void Initialize()
{

```

```

if (NetworkManager.Singleton.IsServer)
{
    currentStageIndexNet.Value = 0;
}

var predictionStage = gameObject.GetComponent<PredictionStage>();
var playerTurnStage = gameObject.GetComponent<PlayerTurnStage>();
var resultStage = gameObject.GetComponent<ResultStage>();
var bossTurnStage = gameObject.GetComponent<BossTurnStage>();

predictionStage.Initialize(this);
playerTurnStage.Initialize(this, PlayerTurnTime, timerText, confirmationText,
confirmationButton);
resultStage.Initialize(this);
bossTurnStage.Initialize(this);

_stages = new List<GameState> { predictionStage, playerTurnStage, resultStage,
bossTurnStage };
}

private void StartBattle()
{
    if (NetworkManager.Singleton.IsServer)
    {
        NextStage(0);
    }
}

private void LoseBattle()
{

```

```
isCombatRunning = false;
LoseMenu.SetActive(true);
Debug.Log("Combat Ended. You Lose.");
}
```

```
private void WinBattle()
{
    isCombatRunning = false;
    WinMenu.SetActive(true);
    Debug.Log("Combat Ended. You Win.");
}
```

```
private void NextStage(int stageIndex)
{
    if (currentStage != null)
    {
        currentStage.Exit();
    }
```

```
    if (isCombatRunning)
    {
        currentStage = _stages[stageIndex];
        currentStageIndex = stageIndex;
        currentStage.Enter();
    }
}
```

```
public void TransitionToNextStage()
{
    if (NetworkManager.Singleton.IsServer)
```

```

    {
        int nextStageIndex = (currentStageIndexNet.Value + 1) % _stages.Count;
        currentStageIndexNet.Value = nextStageIndex;
        NextStage(nextStageIndex);
    }
}

```

```

using System.Collections;
using System.Collections.Generic;
using TMLPro;
using Unity.Netcode;
using UnityEngine;

```

```

public abstract class GameState : NetworkBehaviour
{
    protected CombatStageManager gameStateManager;

    public override void OnNetworkSpawn()
    {
        base.OnNetworkSpawn();
    }

    public abstract void Enter();

    public abstract void Exit();

    public abstract void UpdateStage();
}

```

```

using System.Collections;

```

```

using System.Collections.Generic;
using System.Linq;
using Unity.Netcode;
using UnityEngine;
using static MoreMountains.Tools.ShaderController;

public class PredictionStage : GameState
{
    private          BossManager          bossManager          =>
    GameObject.FindFirstObjectByType<BossManager>();
    private          CombatPlayerDataInStage      combatPlayerDataInStage      =>
    FindFirstObjectByType<CombatPlayerDataInStage>();
    private          PlayerInfoData          playerInfoData          =>
    GameObject.FindFirstObjectByType<PlayerInfoData>();
    private int localId;

    public void Initialize(CombatStageManager manager)
    {
        gameStateManager = manager;
        GlobalEventSystem.BossEndCombo.AddListener(EndTurn);

        GlobalEventSystem.TargetPointsForActionsChooosed.AddListener(StartNewBossCo
mbo);

        localId = playerInfoData.PlayerIDThisPlayer;
    }

    public override void Enter()
    {
        if (NetworkManager.Singleton.IsServer)

```

```

{
    bossManager.ChoiceCombo();
}

GlobalEventSystem.SendPredictionStageStarted();

//Debug.Log("Entering Prediction Stage");
RestoreEnergy();
ShieldsRemove();
FeelFeedbacksManager.instance.SetActivePredictionEffect(true);

}

private void StartNewBossCombo()
{
    BossManager.instance.CastCombo();
}

public override void Exit()
{
    GlobalEventSystem.SendPredictionStageEnded();
    FeelFeedbacksManager.instance.SetActivePredictionEffect(false);
}

public override void UpdateStage()
{
    if (NetworkManager.Singleton.IsServer)
    {

```

```

    }
    else
    {

    }
}

private void RestoreEnergy()
{
    for (int i = 0; i < playerInfoData.PlayerCount; i++)
    {
        if (combatPlayerDataInStage._TotalStatsList[i] != null)
        {
            int newEnergy =
combatPlayerDataInStage._TotalStatsList[i].currentCombat.CurrentEnergy +
combatPlayerDataInStage._TotalStatsList[i].general.EnergyPerTurn;
            if (newEnergy >
combatPlayerDataInStage._TotalStatsList[i].general.MaxEnergy)
ChangePlayerEnergyRpc(combatPlayerDataInStage._TotalStatsList[i].general.MaxE
nergy, localId);
            else ChangePlayerEnergyRpc(newEnergy, i);
        }
    }
    GlobalEventSystem.SendEnergyChange();
}

[Rpc(SendTo.ClientsAndHost)]
private void ChangePlayerEnergyRpc(int newEnergy, int id)
{
    newEnergy = EnergyUpdate(newEnergy, id);
}

```

```

        combatPlayerDataInStage._TotalStatsList[id].currentCombat.CurrentEnergy =
newEnergy;
    }

    private int EnergyUpdate(int newEnergy, int playerID)
    {
        Debug.Log(newEnergy);

        int energyReductionID =
DBManager.instance.GetEffectDataList().FirstOrDefault(e => e.EffectScript is
EnergyChanger).ContentID;

        var targetEffectList = EffectsManager.instance.GetEffectList(new
HeroCombatObject(playerID, CombatPlayerDataInStage.instance));
        if (targetEffectList != null && targetEffectList.ContainsKey(playerID))
        {
            var energyReduction = targetEffectList[playerID].Effects
                .FirstOrDefault(e => e.ContentID == energyReductionID);

            if (energyReduction != null)
            {
                newEnergy += energyReduction.EffectScript.currentStacks;
            }
        }
        Debug.Log(newEnergy);
        return newEnergy;
    }

    private void EndTurn()
    {
        if (CombatStageManager.instance.currentStage is PredictionStage)
gameStateManager.TransitionToNextStage();
    }

```

```

    }

    private void ShieldsRemove()
    {
        bossManager.bossStats.CurrentShield = 0;
        List<AllPlayerStats> playerStats =
        combatPlayerDataInStage._TotalStatsList.ToList();
        for (int i = 0; i < playerStats.Count; i++)
        playerStats[i].currentCombat.CurrentShield = 0;
        GlobalEventSystem.SendPlayerShieldChanged();
    }
}

```

```

using System;
using System.Collections;
using System.Collections.Generic;
using TMLPro;
using Unity.Collections.LowLevel.Unsafe;
using Unity.Netcode;
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.UI;

```

```

public class PlayerTurnStage : GameState
{
    [SerializeField] float timeBetweenNewBossCombo;

    private float maxTurnTime;
    private TextMeshProUGUI timerText;

```

```

private NetworkVariable<float> remainingTime = new
NetworkVariable<float>(0f);

private NetworkVariable<int> playersConfirmed = new NetworkVariable<int>(0);
private TextMeshProUGUI confirmationText;
private Button confirmationButton;
private Coroutine timer;
private InputActions inputActions;
private bool endingTurn = false;

private PlayerInfoData playerInfoData =>
GameObject.FindFirstObjectByType<PlayerInfoData>();

private CombatPlayerDataInStage combatPlayerDataInStage =>
FindFirstObjectByType<CombatPlayerDataInStage>();

private GUIDataBase dataBaseGUI =>
GameObject.FindFirstObjectByType<GUIDataBase>();

public void Initialize(CombatStageManager manager, float maxTurnTime,
TextMeshProUGUI timerText, TextMeshProUGUI confirmationText, Button
confirmationButton)
{
    inputActions = new InputActions();
    inputActions.Combat.EndPlayerTurn.performed += _ => ConfirmEndTurn();
    gameStateManager = manager;
    this.maxTurnTime = maxTurnTime;
    this.timerText = timerText;
    this.confirmationText = confirmationText;
    this.confirmationButton = confirmationButton;
    this.confirmationButton.onClick.AddListener(ConfirmEndTurn);
    playersConfirmed.OnValueChanged += UpdateConfirmationUI;

```

```

GlobalEventSystem.BossEndCombo.AddListener(StartNewBossComboAfterTime);
    dataBaseGUI.EndTurnButtonActiveChange(false);
    dataBaseGUI.TimerActiveChange(false);
}

```

```

public override void Enter()
{
    endingTurn = false;
    //Debug.Log("Enter Player Turn Stage");
    if (NetworkManager.Singleton.IsServer)
    {
        remainingTime.Value = maxTurnTime;
        playersConfirmed.Value = 0;
        timer = gameStateManager.StartCoroutine(Timer());
    }
    else Invoke(nameof(StartNewBossCombo), timeBetweenNewBossCombo);
    inputActions.Enable();

    EnablePlayerTurnUI(true);
    UpdateConfirmationUI(0, playersConfirmed.Value);
    GlobalEventSystem.SendPlayerTurnStageStarted();
}

```

```

public override void Exit()
{
    GlobalEventSystem.SendPlayerTurnStageEnded();
    EnablePlayerTurnUI(false);
    inputActions.Disable();
}

```

```

public override void UpdateStage()
{
    if (NetworkManager.Singleton.IsServer)
    {
        if (playersConfirmed.Value >=
combatPlayerDataInStage.CountOfAlivePlayers())
        {
            StartEndingTurn();
            gameStateManager.StopCoroutine(timer);
        }
    }
}

private void StartNewBossComboAfterTime()
{
    if (CombatStageManager.instance.currentStage is PlayerTurnStage &&
!endingTurn) Invoke(nameof(StartNewBossCombo),
timeBetweenNewBossCombo);
    else if (CombatStageManager.instance.currentStage is PlayerTurnStage &&
endingTurn) EndTurn();
}

private void StartNewBossCombo()
{
    BossManager.instance.CastCombo();
}

private void UpdateConfirmationUI(int previousValue, int newValue)
{

```

```

        if (confirmationText != null)
        {
            confirmationText.text = newValue + "/" +
combatPlayerDataInStage.CountOfAlivePlayers();
        }
    }
}

```

```

private IEnumerator Timer()
{
    UpdateTimerUIRpc();
    while (remainingTime.Value > 0)
    {
        yield return new WaitForSeconds(1f);

        remainingTime.Value -= 1f;

        UpdateTimerUIRpc();

        if (remainingTime.Value <= 0)
        {
            StartEndingTurn();
            yield break;
        }
    }
}

```

```

public void ConfirmEndTurn()
{
    //Debug.Log("ConfirmEndTurn");
    inputActions.Disable();
}

```

```
ConfirmEndTurnRpc();
```

```
dataBaseGUI.EndTurnButtonActiveChange(false);
```

```
GlobalEventSystem.SendPlayerTurnEndConfirmed();
```

```
}
```

```
[Rpc(SendTo.Server)]
```

```
private void ConfirmEndTurnRpc()
```

```
{
```

```
    playersConfirmed.Value += 1;
```

```
}
```

```
[Rpc(SendTo.ClientsAndHost)]
```

```
private void UpdateTimerUIRpc()
```

```
{
```

```
    UpdateTimerUI();
```

```
}
```

```
private void UpdateTimerUI()
```

```
{
```

```
    if (timerText != null)
```

```
    {
```

```
        timerText.text = $"{remainingTime.Value}";
```

```
GlobalEventSystem.SendPlayerTurnStageTimerUpdate(remainingTime.Value);
```

```
}
```

```
}
```

```

private void EndTurn()
{
    gameStateManager.TransitionToNextStage();
    EnablePlayerTurnUI(false);

    GlobalEventSystem.SendPlayerTurnEndConfirmed();
}

private void StartEndingTurn()
{
    if (!endingTurn)
    {
        //Debug.Log("StartEndingTurn");
        EnablePlayerTurnUI(false);
        GlobalEventSystem.SendPlayerTurnEnding();
        inputActions.Disable();
        endingTurn = true;
    }
}

private void EnablePlayerTurnUI(bool enable)
{
    if (combatPlayerDataInStage.aliveStatus[playerInfoData.PlayerIDThisPlayer])
    {
        dataBaseGUI.EndTurnButtonActiveChange(enable);
        //dataBaseGUI.SkillButtonListActiveChange(enable);
    }
    dataBaseGUI.TimerActiveChange(enable);
}
}

```

```
using Unity.Netcode;
```

```
using UnityEngine;
```

```
public class ResultStage : GameState
```

```
{
```

```
    [SerializeField] private float TimeBeforeFirstPlayerMovement = 1f;
```

```
    [SerializeField] private float TimeBeforeFirstPlayerCast = 1f;
```

```
    [SerializeField] private float TimeBetweenPlayerMovement = 1f;
```

```
    private NetworkVariable<int> PlayerEndMovind = new();
```

```
    private NetworkVariable<int> PlayerEndResultTurn = new();
```

```
    private          PlayerInfoData          playerInfoData          =>
```

```
FindFirstObjectByType<PlayerInfoData>());
```

```
    private int playerCount;
```

```
    private int _orderInTurnPriority;
```

```
    private bool EndMoving = false;
```

```
    public void Initialize(CombatStageManager manager)
```

```
{
```

```
        gameStateManager = manager;
```

```
        playerCount = playerInfoData.PlayerCount;
```

```
GlobalEventSystem.PlayerEndMoving.AddListener(ConfirmPlayerEndMoving);
```

```
GlobalEventSystem.PlayerEndResultTurn.AddListener(ConfirmPlayerEndResultTurn);
```

```
}
```

```

private void ConfirmPlayerEndMoving(int orderInTurnPriority)
{
    _orderInTurnPriority = orderInTurnPriority + 1;
    SendResultStageStartedAfterTime();
    ConfirmPlayerEndMovingRpc();
}

private void SendResultStageStartedAfterTime()
{
    Invoke(nameof(SendStartResultStageForPlayer),
TimeBetweenPlayerMovement);
}

private void SendStartResultStageForPlayer()
{
    SendStartResultStageForPlayerRpc(_orderInTurnPriority);
}

[Rpc(SendTo.ClientsAndHost)]
private void SendStartResultStageForPlayerRpc(int orderInTurnPriority)
{
    GlobalEventSystem.SendStartResultStageForPlayer(orderInTurnPriority);
}

private void ConfirmPlayerEndResultTurning(int orderInTurnPriority)
{
    //Debug.Log("ConfirmPlayerEndResultTurning");
    SendAllPlayersEndMovingRpc(orderInTurnPriority + 1);
    ConfirmPlayerEndResultTurnRpc();
}

```

```

[Rpc(SendTo.ClientsAndHost)]
private void SendAllPlayersEndMovingRpc(int orderInTurnPriority)
{
    if (orderInTurnPriority == 0) GlobalEventSystem.SendAllPlayersEndMoving();
    GlobalEventSystem.SendStartCastPlayer(orderInTurnPriority);
}

public override void Enter()
{
    //Debug.Log("Enter Result Stage");

    if (NetworkManager.Singleton.IsServer)
    {
        PlayerEndMovind.Value = 0;
        PlayerEndResultTurn.Value = 0;
        Invoke(nameof(SendStartResultStageForFirstPlayer),
TimeBeforeFirstPlayerMovement);
    }

    GlobalEventSystem.SendResultStageStarted();
}

private void SendStartResultStageForFirstPlayer()
{
    SendStartResultStageForPlayerRpc(0);
}

public override void Exit()
{

```

```

GlobalEventSystem.SendAllPlayersEndMoving();
GlobalEventSystem.SendResultStageEnded();
//Debug.Log("Exiting Result Stage");
}

public override void UpdateStage()
{
    if (PlayerEndMovind.Value >= playerCount && !EndMoving)
    {
        EndMoving = true;
        if (NetworkManager.Singleton.IsServer)
        Invoke(nameof(SendStartCastPlayer), TimeBeforeFirstPlayerCast);
        //Debug.Log("PlayerEndMovind.Value >= playerCount");
    }

    if (NetworkManager.Singleton.IsServer)
    {
        if (PlayerEndResultTurn.Value >= playerCount)
        {
            //Debug.Log("PlayerEndResultTurn.Value >= playerCount");
            EndTurn();
        }
    }
}

private void SendStartCastPlayer()
{
    SendAllPlayersEndMovingRpc(0);
}

```

```

[Rpc(SendTo.Server)]
private void ConfirmPlayerEndMovingRpc()
{
    PlayerEndMovind.Value += 1;
}

[Rpc(SendTo.Server)]
private void ConfirmPlayerEndResultTurnRpc()
{
    PlayerEndResultTurn.Value += 1;
}

private void EndTurn()
{
    if (NetworkManager.Singleton.IsServer)
    {
        gameStateManager.TransitionToNextStage();
        PlayerEndMovind.Value = 0;
        PlayerEndResultTurn.Value = 0;
    }
    EndMoving = false;
}
}

using System;
using System.Collections;
using System.Collections.Generic;
using TMLPro;
using Unity.Netcode;
using UnityEngine;

```

```

public class BossTurnStage : GameState
{
    [SerializeField] float TimeBetweenRounds = 1f;
    private          BossManager          bossManager          =>
GameObject.FindFirstObjectByType<BossManager>());

    public void Initialize(CombatStageManager manager)
    {
        gameStateManager = manager;
        GlobalEventSystem.BossEndCombo.AddListener(EndingTurn);
    }

    public override void Enter()
    {
        if (NetworkManager.Singleton.IsServer && gameStateManager.IsSpawned)
        {

        }

        GlobalEventSystem.SendBossTurnStageStarted();

        bossManager.CastCombo();
    }

    public override void Exit()
    {
        GlobalEventSystem.SendBossTurnStageEnded();
        //Debug.Log("Exiting Boss Turn Stage");
    }
}

```

```

    }

    public override void UpdateStage()
    {
        if (NetworkManager.Singleton != null)
        {
            if (NetworkManager.Singleton.IsServer)
            {

            }

        }
    }

    private void EndingTurn()
    {
        if (CombatStageManager.instance.currentStage is BossTurnStage)
        Invoke(nameof(EndTurn), TimeBetweenRounds);
    }

    private void EndTurn()
    {
        gameStateManager.TransitionToNextStage();
    }

}

```